teams are generally better, while the ability to influence priorities is reduced. There is no silver bullet; a specific reporting level decision should be made in the context of suitability and specific needs of an organization. There are, a few thumb rules, however:

- SQA should not report to the project manager
- SQA should report to somewhere within the local software labs or plant organization
- Typically, there should be no more than one management position between SQA and the senior location manager
- SQA should always have a "dotted-line" relationship to a senior corporate quality executive
- Wherever possible, SQA should report to someone who has a vested interest in software quality

2.4.6 *Launching the SQA Function in an Organization*

Quality mission cannot be accomplished without the solid support from the top management. Therefore, the first essential step in establishing an SQA function is to secure top management agreement on its goals. Since it is the senior management who ultimately helps resolve all escalated issues, it is important that their agreement is taken in advance on the basis for doing so. If this is not done, then SQA function is unlikely to get the support from the senior management in situations of disputes. In view of this, following are some important points to keep in mind while announcing and launching the SQA Function in an organization:

- Initiate the SQA Program
- Identify SQA issues well in advance
- Write the SQA Plan
- Establish standards
- Establish the SQA function
- Conduct training to create awareness about the SQA function and promote the SQA function
- Implement the SQA Plan
- Evaluate the SQA Program

2.4.7 *Commitment By Top Management*

In the previous section, we already started this discussion; we'll complete it in this section. By now, it would be clear that the cause of quality cannot be simply delegated to a 'quality champion' or project leaders. While all mangers and staff must own and take responsibility for day-to-day quality issues with their control, the quality endeavour must be supported, endorsed and enforced from the top, as part of "quality

culture". Without this, attempts to improve quality will fail, since staff will feel demoralized in absence of solid support form top management. A quality champion can be appointed by all means to do this Herculean task but the role of such a champion is only to help top management fulfil their responsibility to quality. Top management can best fulfil this responsibility by:

- Treating quality as a strategic business issue (remember the discussion in Chapter 1)
- Developing a policy for quality; a policy that can be implemented
- Taking customers in confidence to take their buy-in, educating the customer about the importance of quality
- Ensuring the quality policy is communicated to all the employees and is understood by them
- Ensuring that the Quality policy is implemented throughout the organization
- Regularly reviewing the quality policy and its implementation

2.4.8 People Issues in Software Quality Assurance

Quality is a matter of the mind-set and "attitudes". In all the immediately preceding sections we've drawn attention to the people aspect of SQA. In this section, we'll specifically focus on this topic considering its importance. Successful SQA function implementation requires empowering employees. Staff will not be able to produce quality software and give their support to SQA function unless they have the necessary skills, knowledge, experience, and competence. In the first place, Management must select the right staff for the SQA job and must provide staff with the resources and training they need to give their best. This will motivate staff, and generate a commitment to excellence and continuous improvement. Under these circumstances, authority and decision making can be delegated to the appropriate level, where they can be exercised most effectively to respond promptly to customers' need.

In section 2.4.3, we also discussed, the skillfulness needed in performing the SQA role. People-handling skills along with adequate technical background is a combination that makes an SQA successful. Unfortunately, it is a rare combination to come by! Getting good people into SQA is one of the most difficult challenges faced by organizations. The practice of starting new hires or relatively junior employees in the SQA function can work to be effective only with adequate handholding and senior members' involvement in mentoring. Considering the speed of software deliveries today and the ever-shortening life cycles of software projects today, this seems unlikely. An organization can induct younger members into SQA function for grooming them further, provided there are already some seniors to do this noble job. At times, job-rotation schemes (whereby project members work as SQAs for a

few months) can also be effective, but unfortunately, projects will have a penchant to generally transfer their poorer performers to the SQA function. Also, the ability of the SQA function to attract practitioners on job rotation basis will really depend upon the importance of the SQA function "perceived" by the employees. For getting respect for the SQA function only senior management is responsible. One solution that may work is to require that all new development managers be promoted from SQA. This would mean that potential managers would spend about six months to a year in SQA before being promoted into management back in their home departments. While this may look like an extreme measure, it can be effective.

2.4.9 *Total Quality Management*

A final thought before concluding this chapter; the approach to management of quality described in this chapter is often known as Total Quality Management (see the vignette below). Readers can also refer to [5] quoted at the end of this chapter.

> *Total Quality Management is a structured system for satisfying internal and external customers and suppliers by integrating the business environment, continuous improvement, and breakthroughs with development, improvement, and maintenance cycles while changing organizational culture.*

It encourages management to give as much attention to the quality culture of the organization as to its customers. If the internal customers' requirements are agreed and met, a chain of quality is made that reaches out to the external customer. To get people to identify the internal customers for the main output of the work group is to make a start at a total quality process and to sustain the spirit of "continuous improvement". This is important to touch off a dialogue between the "internal supplier" and the "internal customer" that leads them to agree customer requirements in the beginning of creating a "total quality culture". You'll never achieve quality externally until you have quality internally!

REFERENCES

[1] *The Practical Guide to Structured Systems Design*—Meilir Page-Jones (Prentice-Hall International Edition)
[2] *ISO 9000: Preparing for Registration*—James L. Lamprecht (ASQC Quality Press)
[3] *Quality Function Deployment*—Nina Godbole (Article in the August 2001 issue of the Information Technology Magazine
[4] *Effective Project Management*—Robert Wysocki (John Wiley & Sons)
[5] Management in a Quality Environment—David N.Griffiths (TATA McGraw-Hill)

Chapter 3

PLANNING FOR SOFTWARE QUALITY ASSURANCE

3.1 Software Quality Assurance Plans

3.1.1 Purpose of SQA Plan

In section 2.4.6 of Chapter 2, we discussed about launching the SQA function in an organization. In section 2.2 of Chapter 2, we talked about what the various organizational players expect from the Quality Management System. Users' expectation from software is usually more than the expectation that it will work. Software must pass the Acceptance Test. When custom software is built for the specific requirements of a particular customer, a series of *'acceptance tests'* are conducted to enable the customer to *validate* all requirements.

Failed software entails loss of time and the lost competitive advantage. Therefore, while placing the contract on the supplier, user is entitled to ask for an assurance that the software, when finished, will perform according to its specification. The public at large may be affected by the user's use of the software. The user's public liability for damages, should the public be harmed in some way by the malfunctioning of the software, reinforces the user's requirement that the software should be of suitable quality. Thus Quality Assurance does have a dimension of social commitment, too.

Quality is achieved by building it into the software all throughout the project lifecycle; it cannot be added at the end of software development. This implies that the relationship between software quality and the software development process is known and that, by proper engineering of this process, a product of the appropriate quality can be produced. Defining the standard software process is discussed in a later section of this chapter. The quality of the software as it is developed must be continually evaluated to ensure that it is satisfactory. This evaluation must be planned and documented and made known to those who execute the software development project. The planning of how

ality is to be built in and also how it is to be evaluated is recorded in e *quality assurance plan*. Quality Plan should be produced very early n the life of the software development project. It has a bearing on the *quality objectives* of the organization. It may not exist as a separate document; it could be incorporated into project's software development plan. The software quality plan should provide confidence to the user that the software product or service will be of high quality (as agreed between the Customer and the Supplier).

3.1.2 Content of the Software Quality Assurance Plan

Each development and maintenance project should have a Software Quality Assurance Plan that specifies its goals, the SQA tasks to be performed, the standards against which the development work is to be measured, and the procedures and organizational structure.

The IEEE Standard for Software Quality Assurance Plans states that the plan should contain the following sections:

- Purpose
- Reference documents
- Management
- Documentation
- Standards, practices and conventions
- Reviews and Audits
- Configuration Management
- Problem reporting and corrective action
- Tools, techniques and methodologies
- Code Control
- Media Control
- Supplier Control
- Records collection, maintenance and retention

At the end of this section, we've illustrated how a quality objective section of a project plan would be written by showing an excerpt from a real life project.

3.1.2.1 Purpose

This states the specific purpose and scope of the SQA plan. It names the software products that it covers and describes the use to which they will be put.

3.1.2.2 Referenced documents

A complete list of the documents referenced in the plan.

3.1.2.3 Management

IEEE standard lays down three aspects that should be covered in this section of the Quality Assurance Plan: *organization, tasks* and *responsibilities*

Organization

This section is about the project organization, roles of team members, their hierarchy etc. It is important that the head of Software Quality Assurance (SQA) function in the organization has adequate authority to be able to perform independent verification that the processes (as laid down in the Quality Management System of the organization) are adhered to.

Tasks

Assuming the prior existence of a comprehensive set of company standards, the chronological sequence of tasks, which need to be performed, includes:

- Preparing preliminary software requirements specification; perhaps as a part of the development of a system involving hardware and software
- Preparation of a software configuration plan, a software quality assurance plan and a software development plan which may or may not include the other two documents
- Software requirements review
- Software design review
- Preparation of software requirements specification
- Conducting review of software requirements specification document
- Preparing a software test plan
- Preparing a top-level software design (Also known as High Level Design)
- Preparing a draft support documentation, e.g. user manuals etc.
- Top level software design review
- Preparing software test description
- Production of a detailed software design
- Detailed software design review
- Production of software test procedures
- Production of source code and object code for the code units
- Testing of code units
- Integration of software modules/units
- Testing of integrated software units
- Systems integration and systems integration testing
- Acceptance Testing

Responsibilities

The project manager and design/development teams have primary responsibility for the quality controls applied during the development of the software project. The quality manager will:

- Define the responsibilities of quality personnel in the form of quality assurance procedures applicable to the project (Refer section 2.3 of Chapter 2 for a discussion on what the auditor would look for)
- Agree to the quality plan with the project manager
- Approve the plan of audits for the project which are to be carried out by quality personnel
- Resolve any disagreement between the project manager and quality personnel on matters relating to quality (for example non-conformances raised during internal audits and reviews, classifications of the non-conformances raised, dates for implementing corrective actions etc.)
- Review the activities performed by project personnel to ensure that the requirements of the quality plan and quality procedures are being satisfied
- Review the contents of software standards, engineering codes of practice and quality procedures for adequacy and efficiency

Quality personnel will:

- Carry out planned internal audits of the project to assess compliance with quality objectives
- Agree on corrective action with the project manager for any discrepancies, non-conformities found and ensure that corrective action is taken
- Evaluate defect trends and take appropriate action
- Refer any unresolved discrepancies to the quality manager for resolution

In the section of the Software Quality Assurance Plan, one would be looking for a real commitment to quality on part of the company. (Refer section 2.4.7 of Chapter 2). Quality personnel must be seen to have authority and power (In reality, some actual issues come up on this. They are discussed in section 3.3 of this chapter). It is desirable that the quality personnel should be independent from the developers of the software. (Refer section 2.4.5 of Chapter 2 for a brief discussion on this) They should have sufficient resources, authority and technical expertise to perform quality evaluation activities objectively (Refer section 2.4.3 of Chapter 2 for a discussion on this) and initiate corrective actions, if required.

3.1.2.4 *Documentation*

The basic purpose of this section of the Software Quality Assurance Plan is to describe the documentation to be produced and how it is to be reviewed. All the documentation relating to the development, verification, use and maintenance of the software will be listed. This section will normally include the following:

- Software Requirements Specification (SRS)
- Software Design Description
- Software Verification Plan: this describes the methods used to verify whether the requirements in the SRS are implemented in the design and in the code, and that the code, when executed, meets the requirements expressed in the SRS. These methods will include Inspection, Reviews and Testing etc. (Refer separate chapters on "Reviews and Inspection" and "Testing")
- Software verification report: this describes the results of executing the software verification plan and includes the results of all reviews, audits, tests etc.
- Reference to Software Standards (ISO, CMM, IEEE etc) and procedures mentioned and defined as in the Quality Manual and Quality Management System respectively of the organi .ation). In section 2.2 of Chapter 2, we discussed what expectations the organizational players have from the Quality Management System.
- User guides, operators' and programmers' manual (This section is optional). In this section you may provide pointers to these manuals instead of directly including them here.
- Configuration Management Plan (for details, see the separate chapter on Software Configuration Management.) If your project is very large with complex configuration management activities, then you can have a separate document on Software Configuration Management; here you may only provide a reference to that.
- Software Quality Objectives: In this section, you define the various measures and metrics for the quality objective suitable for you plan along with the Upper and Lower Control limits for this.

3.1.2.5 *Standards, Practices and Conventions*

This section of the Software Quality Assurance plan should contain at a minimum, the following:

- Documentation standards
- Logic structure standards
- Coding standards
- Commentary standards

The developer will have designed and documented the process by which the software will be built. This expresses the methodology adopted. Work will be undertaken according to defined *standards* and in defined ways. (This is what the procedures in the QMS explain). These standards will be written down and used by the development team. Quality personnel shall check that the work has been done in accordance with these standards. The standards encompassed in these documents will cover things as diverse as the contents and format of each of the documents produced during the development. (These could be as per the Templates provided in the QMS.) The standard could also mention the textual layout of the code, the reviews to be undertaken (QMS may define what kind of reviews are mandatory depending on the nature of the project i.e. full development project, or an enhancement project or a purely maintenance work project etc.), the composition of the review board, the design methodology to be used to produce the software design, configuration management and testing.

Thus, this section of the SQA plan will identify the standards, practices and conventions to be applied and state how it is proposed to ensure that the work is undertaken in accordance with these standards. The plan states that, at a minimum, there will be standards for documentation, algorithm description, coding and ways of adding comment lines in a software program.

3.1.2.6 Reviews and Audits

This section of the Software Quality Assurance plan will state which technical and managerial reviews will be undertaken and how they will be carried out. Project can separately keep a schedule of provisional dates for conducting these reviews. The ANSI standard (American National Standards Institute) suggests that the following would be a minimum set of reviews:

Software Requirements Specification Review

This review is held to approve the document defining the software requirements specification and it aims to check the adequacy of the requirements. At this review the project manager will be expected to state which evaluation activities have taken place during the preparation of the document. These evaluation activities will ensure that the preparation of the requirements specification was undertaken in accordance with the developers' codes of practice using the approved tools and techniques. The document will have been checked to ensure that it conforms to the appropriate organization standard. The technical adequacy of the specification should have been evaluated to ensure that it forms the basis from which software of the desired quality can be developed. The requirements specification document will have been checked for internal consistency, understandability and completeness.

(one good reference for standard practices in drawing requirements specification is the *IEEE Guide to Software Requirements Speciation – ANSI/ IEEE Std 830*)

It is being realized how important it is to ensure that the required quality aspects of the product are explicitly stated in the specification. It is not reasonable to hope that software of the required quality can be produced without proper analysis of the required quality and its formal explicit statement. The adequacy of the quality aspects can be evaluated once they have been stated. Proper recognition of the desired quality aspects at the earliest possible stages is in everyone's interest. To aid traceability, it is helpful if each requirement has a unique identifier. (Refer to discussion under section 1.5 of Chapter 1)

Preliminary Design Review

The purpose of this review is to approve formally, the software top-level design document. As part of this process a summary of the quality evaluation, activities undertaken during the top-level design will be reviewed. These will be included ensuring that:

- The design was produced in accordance with the development standards chosen to implement the selected methodology
- All the necessary tasks were undertaken (Joint Application Review, Proof of Concept etc)
- The top-level design is an adequate basis for future work
- The top-level design, when implemented, will satisfy any sizing and timing constraints
- The software top-level design document was produced in accordance with the organization's standard and is internally consistent, understandable, complete and appropriately detailed

The design document will be checked to ensure that each of the requirements in the requirements specification can be traced to a part of the design. Forward and backward traceability is extremely important. (In fact this concept of *Bi-directional Traceability* is emphasized by the CMM-Integrated) It should be possible to trace each requirement forward from the requirements specification and see how it is implemented at each stage of the development process. Similarly, it should be possible to take any part of the software product at any stage in the production process and trace the reason for its existence back to the implementation of a particular requirement.(Refer section 2.3.1 of Chapter2)

Critical Design Review

The purpose of this review is to approve the software detailed design document as a basis for further development work. As part of the process, a summary of the software evaluation activities undertaken

during the detailed design phase will be reviewed. These will be included ensuring that:

- The design was undertaken in accordance with the organization's standards and is technically feasible
- All the necessary tasks have been undertaken (Application architecture, entity relationship diagrams, entity life history diagrams, state transition diagrams, principles of Coupling and Cohesion etc)
- The detailed design was internally consistent, understandable, complete and appropriately detailed
- Traceability is maintained through the top-level design to the software requirements specification
- Test cases for unit test and integration test have been prepared as part of the design and have been checked for consistency with the organization's standards

Software Verification Review

The purpose of this review if to approve the test plan. It is an evaluation of the adequacy and completeness of the methods described.

Functional Audit

This is held to verify that all the requirements in the software requirements specification have been met.

Physical Audit

This is held to verify that the software and its documentation are internally consistent prior to delivery to the user.

In-Process Audits

In-process audits of a sample design are held to verify the consistency of the design. Points, which will be checked, are consistency of code and documentation, consistency of design and functional requirements and consistency of fu ctional requirements and test descriptions.

Management Reviews

It is important that the execution of the quality plan is evaluated and there will be one or more reviews of this. For example, if it is found that there is a slippage (i.e crossing of either the Upper Control Limit or the Lower Control Limit) of the agreed and documented goals of the project, then causal analysis must be done (Refer Figure 3.2 for an example) to find out the reasons for slippage, decide on an action plan to prevent this in future, inform the management and track the action plan (produced by the project team) to closure.

3.1.2.7 Configuration Management

We'll undertake detailed discussion about this in a separate chapter dedicated to this topic. This section of the Software Quality Assurance plan will cover **configuration identification, configuration control, configuration status accounting,** and **configuration auditing.** As said earlier, this section of the SQA plan may refer to a separately written configuration management plan in case of large complex software projects. Various forms, formats and templates in relation to configuration management will be discussed in the chapter on this topic.

3.1.2.8 Problem Reporting and Corrective Action

This section of the Software Quality Assurance plan will describe the system, which ensures that software problems are documented and resolved. It should be a closed-loop system. All the problems should be promptly reported/escalated at appropriate level, acted upon and resolved. Each problem should be capable of being tracked throughout life cycle of the software project. Each problem should be analyzed to determine its significance and causes and classified by category (such as requirements, design, coding etc.) and each problem must have severity level and a priority number (to enable action on it). Trends in the problems reported should be identified.

For each problem or adverse trend discovered, some corrective action and a target completion date should be identified. The appropriate level of management should be made aware of the problems and adverse trends. Corrective action will be authorized and the activities undertaken will be documented. The corrective action taken will be evaluated to ensure that it solved the problem without introducing any new problems. Management should monitor the status of all unresolved problems and be aware of any that have not been resolved by their target completion date.

3.1.2.9 Tools, Techniques and Methodologies

This section of the SQA plan should identify the special software tools, techniques and methodologies employed that support quality assurance, state their purposes and describe their uses.

3.1.2.10 Code Control

In a software project, this is likely to be implemented in conjunction with the *library function.* The library receives and maintains copies of all software tools and documentation. The library will issue all material and ensure that the most recently Authorized Version is the one routinely available. Access to code files is controlled to ensure that no

unauthorized use or modification takes place. The library will ensure that the correct version of software is submitted for testing. A large project will require a full time library function to take care of distribution of the *configuration items* (more about this in the chapter on Software Configuration Management).

3.1.2.11 Media Control and Back Up

This section of the Software Quality Assurance Plan will describe how the media are to be protected from unauthorized access or damage. One would expect to see details of storage arrangements that protect the media from harmful environmental conditions. At least one back-up copy of the current configuration should be held safely in a remote off-site location. Actually, *Disaster Recovery Plan* is a much larger aspect of Media Control and back up. It is a very large topic on its own. Security threats to a software project come from the following environmental factors:

- Fire Damage
- Water Damage
- Energy Variations
- Structural Damage
- Pollution
- Unauthorized Intrusion
- Viruses and Worms
- Misuse of Software, Data and Services

The purpose of a disaster recovery plan is to enable the project or information systems function to restore operations in the event of some type of disasters (natural like storms, floods etc. or man-made). In some organizations, periodic surveys are conducted to assess the adequacy of their disaster recovery plan. A common recurring finding is that the quality of the disaster recovery plans— if at all an organization has one, often, it is low quality. In organizations that have extensive decentralization and distributed computing resources, disaster recovery planning will be an onerous activity. The minimum that an SQA plan expects is media control and a suitable scheme for back up as discussed above.

3.1.2.12 Supplier Control

This has relevance while outsourcing some components of a software project. It is important that externally developed software is of the appropriate quality. From the CMM-I angle, its importance comes from the fact that "Supplier Agreement Management" (SAM) is an all new process area under CMM-I.

Clause 7.4 of ISO 9001:2000 states that: (as part of the caluses)

The organization shall evaluate and select suppliers based on their ability to supply products and services in accordance with the organization's requirements. Criteria for selection, evaluation and re-evaluation shall be established. Records of the results of the evaluation of supplier and any necessary actions arising from the evaluation shall be maintained.

This section of the SQA plan would state how it was supposed to ensure that this was the case. It would normally contain a form of words to the effect that sub-contractors will implement a quality assurance program to the satisfaction of the main contractor who would have the right to evaluate the system. Software received from the subcontractor will of course be tested and evaluated before it is accepted.

It must be acknowledged that there are some cases in which little can be done to influence the quality of bought-in software (re-usable objects, DLLs, Object brokers and similar other components). The classic case concerns operating systems. Anecdotal evidence suggests that in some cases the suppliers have been unwilling to provide any of the assurances concerning quality, support etc. that purchasers have required. Software is supplied on a "take it or leave it" basis, knowing that given the oligopolistic state of the market, the potential purchaser cannot leave it. This situation where the supplier is in the dominant economic position is very difficult to handle and problems that it creates appear incapable of easy resolution.

3.1.2.13 Records Collection, maintenance and Retention

Any successful project will undergo substantial maintenance over a long period. As such, it is important to ensure that all the documentation necessary to undertake this quickly, efficiently and cheaply is going to be available when required. This documentation must be produced during development. It will be retained. This section of the *Software Quality Assurance Plan* will identify who is responsible for ensuring that the appropriate documentation produced and stored, and under what conditions. Important clauses from ISO 9001:2000 pertaining to this are quoted below. This is followed by an illustration of quality assurance plan for a hypothetical project.

ISO 9001:2000 Clause 4.2.3 Control of Documents

A documented procedure shall be established to define the controls needed

a) to approve documents for adequacy prior to issue,
b) to review and update as necessary and re-approve documents,
c) to ensure that changes and the current revision status of documents are identified
d) to ensure that relevant versions of applicable documents are available at points of use,
e) to ensure that documents remain legible and readily identifiable,
f) to ensure that documents of external origin are identified and their distribution controlled, and
g) to prevent the unintended use of obsolete documents, and to apply suitable identification to them if they are retained for any purpose

ISO 9001:2000 Clause 4.2.4 Control of Records

Records shall be established and maintained to provide evidence of conformity to requirements and of the effective operation of the quality management system. Records shall be legible, readily identifiable and retrievable. A documented procedure shall be established to define the controls need for the identification, storage, protection, retrieval, retention time and disposition of records.

Illustration: Quality Assurance Plan of XYZ Project

Quality Objectives
XYZ Project will comply with the agreed quality objectives as given by the QMS. The quality objectives for the project is as follows:

Reviews
Project progress will be reviewed in every week and monthly status report as mentioned in the "Status Reporting section of this document.

Peer Review will be planned & scheduled for each internal or external deliverable as defined in the project plan or in Statement Of Work. Review records will also be maintained as per the approved template.

Test Plan
Test plan for the project is in the document called "Test Plan for XYZ Project". Please refer to the document ID D1003. Since deliverables of each service are documented in Statement Of Work, test plan will be reviewed and updated as and when any new Statement Of Work is being executed or any new service is being started.

Standards & Guidelines
Coding & other project's Standard/Guidelines are documented and kept in the specific area in the controlled space as mentioned in Configuration Management plan. Standard & Guidelines will have to be approved by the Project Manager.

Process Adherence
Project will be executed to comply with the process as defined in this project plan, Contract, Statement Of Work or Standard & Guidelines as mentioned above. It is possible that certain processes (as defined in Quality Management System) are tailored as per the requirement of XYZ project. Adherence to the defined processes should be reviewed in various project reviews and audits.

Document Control
Documents are being categorized in two categories Master Documents & Other Documents. List of all the Master Documents will be kept along with the Master Documents in the specific controlled area as mentioned in Configuration Management Plan. Each Master Document will be issued a running Document ID. Master Documents will be modified as per the process mentioned later in this document while other Documents will be modified as other Configuration Items of this project.

3.2 Software Quality Assurance: Organizational Level Initiatives

3.2.1 *Managing the Software Process*

3.2.1.1 *Process Management*

The organization's software process assets include the Standard Processes, Software Life cycles, Methodologies and Tools, Standards, Tailoring guidelines for project's software process and a library of Best Practices and Project Components. These are created, updated, maintained and made available to projects for use in developing, implementing, and maintaining the projects' defined software processes.

Software development can be extremely complex, with a choice for selecting an alternative from among those available for performing the same task. Organizations aim at reaching a process-oriented way of working rather than being "person dependent". A "defined software process" provides a framework within which the development staff can make choices in an orderly way rather than in an ad-hoc fashion. Such a framework improves communication between staff, allow them to be moved quickly and easily between projects, and makes planning for quality more reliable. At the same time, it needs to be appreciated that software engineering is not an activity that can be rigidly regimented

and structured like a repetitive manufacturing process (although occasionally one hear a term like "software factory" Ref [3]. Therefore, there is a need to trade off the individual project's need for flexibility against the benefits of defined standards and procedures to the organization. In that sense, this organizational level activity is like a closed feedback loop. Its aim is to:

- Define a standard software process: You need a method to define the procedures, to develop and maintain process documentation and a usable set of software process assets.
- Ensure that each project uses an appropriate version of the standard process that has been tailored to its individual needs. For this *"tailoring guidelines"* need to be developed
- Use the results from projects to improve the standard process. For this a *metrics program* needs to be established. (A separate chapter is devoted to discussion on software measurements and metrics).

This is shown in the Figure 3.1. Functions must be established to manage the defined process, and in particular, to carry out process measurement, process improvement, process improvement and technology innovation activities. In large organizations, one or more dedicated teams may discharge these functions. In smaller organizations, a director may be nominated to take care of this; as and when required, teams may be formed to assist the director in this function.

3.2.1.2 Standard Process Definition

Organizations need to establish, document and maintain a standard software process. This standard process may be tailored to suit the need of individual projects as discussed above. It is important for staff and managers involved in the implementation process to participate in the definition of it and to be kept informed about it. Defining a standard software process is a significant task. So it is important to carry out the definition activities in accordance with a documented plan. This plan must set out the activities and schedule for defining the process, specify group and individual responsibilities for the activities, and identify the required resources. It is important for management to review and approve the plan before it is implemented.

The Process for Process Definition should typically address:

- Activities in Process Definition
  - Develop and maintain Organization's software process
  - Identify organization's software process
  - Define /Change Organization's software process
  - Document The Process

  - Review and Approve the Process
  - Release, Distribution and Retirement of QMS Document
- Identification and documentation of Software Life cycles, methods and tools
- Develop and Maintain Tailoring guidelines for projects
- Guidelines for Writing a Process Document
- Guidelines for creating a Release Notice for announcing the new/ modified processes
- Guidelines for reviewing new Process documents
- Guidelines on Process Release

It is important for the standard process to reflect any constraints on working practice that may be imposed by customers, and for it to incorporate up-to-date software engineering methods, and tools. It is also important to ensure that it makes provision for the collection of **project measures** (see section 3.2.1.3). The standard process describes and orders the software tasks that are common to all projects. It also contains guidelines for tailoring the standard process to the needs of different projects, so that each project has its own approved life cycle that defines:

- Required procedures, practices, methods and technologies
- Applicable process and product standards
- Responsibilities, authorities and staff interrelationships
- Required tools and resources
- Process dependencies and interfaces
- Process outputs and completion criteria
- Product and process measurements to be collected.

It is important to set and track quantitative, measurable goals for process improvement, and to direct these goals at increasing the quality of products and services as well as productivity. A process improvement program should be established that empowers staff and managers to improve their own working processes and participate in improvements made by others.

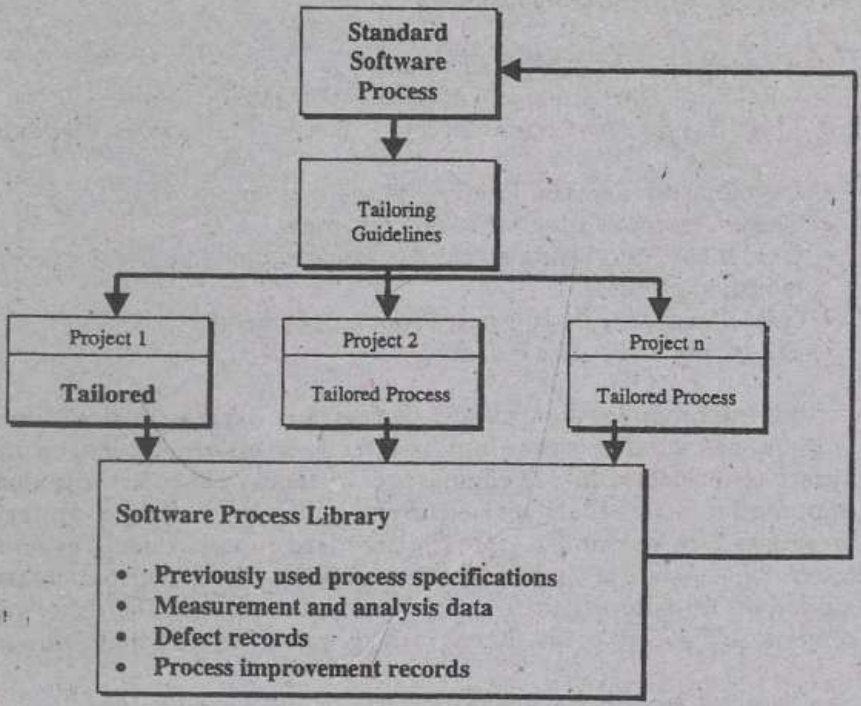

Figure 1: Process Management Activities

3.2.1.3 Software Process Measurements

In the chapter on Software Metrics, there will be a detailed discussion on this topic. Here, only an overview will be provided in the context of software process definition. It is said that you cannot "improve" something that you cannot "measure". Therefore, it is essential to take measurements of the performance of the standard software process, as applied to individual projects. Further, these measurements must be analyzed. (a detailed discussed on 'measurement' and related terms is undertaken in the chapter devoted to this subject.) This analysis should be used to adjust or tune the standard process, with a view to improving it and stabilizing its performance within acceptable limits.

It is important to drive process measurement and analysis activities from a documented and approved plan. The aim of the metrics program is to determine the various parameters to be monitored and measured, define a mechanism for assimilation of data from the implementation team for analysis at Project level as well as at Organization level. A process established for Measurements allows defining a mechanism for collecting various metrics from the projects. The objective is to define measures, to collect and analyze the various measures, to consolidate organizational metrics, monitor and verify process

effectiveness, isolate problem areas and take appropriate actions to ensure that occurrence of such defects is not repeated (*Defect Prevention*) and occurrence of similar defects are minimized. The measurement plan (which is a part of the overall metrics program in the organization) should also describe and schedule the activities to be performed, identify groups and individuals responsible for the activities, specify the required resources, staffing and tools and identify the procedures to be followed.

In developing a metrics program, the following issues need to be resolved:

- "What" should be measured?
- "Why" it should be measured?
- "How" it should be measured?
- "Who" should measure it?
- "When" and "Where" in the process it should be measured

Most important thing is to have a linkage of these measurements to organization's ability to meet the requirements of its customers. If projects are not convinced about this, they are unlikely to provide their support to the organization's metrics program. The selected metrics should therefore:

- be linked to real customer requirements
- support the overall goals and objectives of the measurement program
- support predefined analysis activities
- be consistent across all projects
- cover the entire life cycle, including support and maintenance

Examples of specific measurements that can be used are:

- estimated versus actual SIZE, COST and SCHEDULE data
- quality measurements as defined in the quality plan (PRODUCTIVITY, EFFORT, DEFECT REMOVAL EFFICIENCY, PROCESS COMPLIANCE etc)
- Number and severity of defects in requirements, design and code
- Number and cost of changes to approved requirements and design specifications
- Number and turnaround time of customer requests and bug reports (customer reported bugs)

3.2.1.4 Defect Prevention

Defect prevention is concerned with ensuring that sources of defects that are inherent in the software process, or of defects that occur

repeatedly, are identified and eliminated. Defect prevention activities should be defined and implemented at both organizational and project level. They should be well coordinated.

The objective of Defect prevention at Organizational level is to identify & prioritize defects, which have organization wide impact and prevent them from recurring in future. The defects are identified from following inputs:

- Project wind up reports
- Organizational metrics analysis
- Audit & Assessment reports
- Other organizational level meetings

Defects with organizational wide impact should be identified & prioritized for defect prevention. Root cause analysis should be carried out & recommended preventive actions should be arrived at. Measurements should be made to check the status of defect prevention activities & the status should be presented to senior management for review.

At the project level, it is essential to include defect prevention activities in the project development plan and schedule. One useful technique is to hold meetings during the development to identify defects and analyze their root causes. The defects can then be categorized by cause, such as inadequate training, breakdown of communications, oversight of important details, or manual errors. Figure 3.2 shows an example of a root-cause analysis (also known as the fish bone diagram). The activity of finding out the causes for defects and classification of defects is known as *causal analysis*. (Causal Analysis & Resolution happens to be an important process area at level 5 of CMM-I).

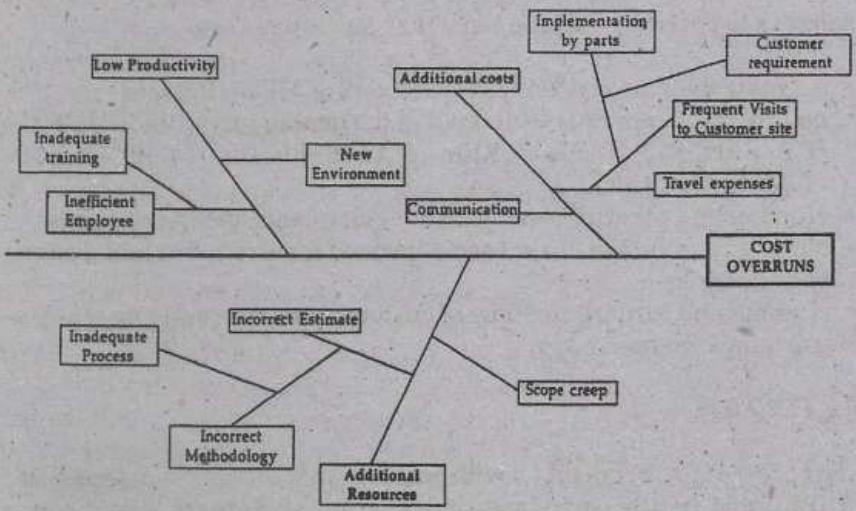

Figure 2: Root Cause Analysis (An Example)

When defects have been identified and classified, measures to prevent their recurrence can be developed and documented. Records of causal analysis meetings should be kept for use by the organization and by the future projects. A format for maintaining a record of Root Cause Analysis is suggested below.

Project Name :	**Team Members Present :**
Project Code :	**Meeting Date :**
Time Spent (Person Hrs)	

Trigger	Where did the trigger occur?	Causes	Corrective Actions Identified	Preventive Action Identified	Impact of Not Preventing	Action Owner	Date for completing Action	Status (OPEN/ CLOSE)	Remarks

Note:

Stage where Trigger occurred:
- Planning
- Requirements
- Design
- Coding
- Testing
- Acceptance

Root Causes:
- P – Process/Methods
- C – Communication
- S – Skill/Training
- T – Tools
- L – Planning
- R – Resources

Impact of not Prev rocess changed.
Effort (Additional / Rework) or
Schedule delay / Post Delivery Defects

Recommendations:
- SP – Successful & PCR can be raised
- SN – Successful
- SF – Successful but needs further study
- PC – Project closed before implementation
- R – Not Successful
- SLO – Successful and Learning Submitted to LDB/PAL,
- SLP – Successful and Learning logged in Project Learnings Document
- SPP – Successful and Project Process changed.

At the organizational level, it is essential to identify and document trends that indicate broad problems across several projects, and to take corrective action to eliminate them. It can be useful to record information on proposed defect prevention actions such as the following: (This is shown in the template shown above)

- description of the defect
- description of the cause
- category of the cause
- stage where the defect was identified
- description of the proposed action
- Whether the proposed action is feasible or not
- person responsible for implementing the action
- date by which the action is to be completed
- description of the areas affected by the corrective action
- individuals who are to be kept informed of the action's status
- date of the next status review
- rationale for key decision

3.2.1.5 Technology Innovation

In section 2.3.16 of Chapter 2 we had discussed about the importance of new technology impact analysis. Here, we discuss Technology Innovation at organizational level. Technology changes encompass a very broad spectrum of changes in the technological arena, in the context of the Quality Management System; it will focus on technology changes or introduction of new technologies that are likely to improve the capability of it. These changes:

- Enable the organization to achieve exacting quality standards resulting in decreases in defects
- Empower it to reduce process cycle times and increase process effectiveness, both of which would in turn improve productivity and quality
- Improve the capability of the organization's standard software processes

Technology change management should be integrated with process change management (the next section), and is compatible with the organizational business directives. Thus, it is essential to monitor and evaluate new technologies from time to time especially in the current era of rapid technology change and obsolescence. Appropriate technologies should be implemented in the organizations. It is therefore important for the organization to develop and maintain a plan for technology innovation. Technology (change) management plan of an organization should define the organization's long term technical

strategy for automating and improving software process activities, and describe its plans for introducing new technologies.

It is essential that a regular feedback be given to staff and senior management on the status of new technologies and the results that have been achieved by their use, and that they are kept aware of relevant areas where new tools and technologies have been used successfully. In large organizations, a separate function/group is established entrusted with the responsibility of technology innovation, coordinating technology innovation activities, providing assistance to projects for piloting new technology etc. A suggested format for recording summary of Technology Pilot is indicated below. As indicated there, it helps to arrive at early estimate of training efforts that may be involved in imbibing the new technology. Various parameters that can be considered in evaluating the new technology are EASE OF AVAILABILITY, EFFORT REQUIRED TO IMPLEMENT TECHNOLOGY, ADDITIONAL EFFORT/ RESOURCES REQUIRED TO USE TECHNOLOGY, VENDOR SUPPORT, and TRAINING REQUIREMENTS.

Technology Pilot Results Summary

Technology piloted
Project Name
Expected Benefits
Cost
Platform on which evaluated
Evaluation period
Modules evaluated
Number of people involved in evaluation
Technology installation effort (person hours)
Training duration (hours/days)
Learning curve (days/months)
Average effort required to use technology during steady state
Benefits (preferably quantitative in terms of schedule, effort, etc.)
Issues/Problems Faced
Lessons Learned
Risks/Uncertainties in results
Recommendations for wider usage in the organization
Additional Comments

3.2.1.6 Process Change Management

Here, we discuss 'Process Change Management' at organizational level. Software CMM (*Capability Maturity Model* has this as *a key process area* at Level 5 of maturity) Organization's standard software process must be maintained under configuration management. For this, it is essential to document and review changes derived from various agencies from time to time. Sources of inputs for process change are:

- Findings and recommendations from Audits and Assessments
- Lessons learnt from monitoring process activities
- Change proposals from project staff and managers
- Analyzed and interpreted processes and product measurement data

A system must be established for employees to communicate the Process Change Request to the Process Engineering Group. The group will then evaluate the change request and accordingly reject or accept the request. The accepted change request will be closed when the changes asked for are incorporated in the process by modifying it and the new process is released. The format for Process Change request should include the following:

- Requester's Name
- Requester's contact no
- Ref to process in the Quality Management System for which change is requested
- Description of change requested
- Justification for the change suggested

For any meaningful process management activity, it is essential that all the accepted process change requests be analyzed to look for various trends. Trends could be - distribution of Process changes requests received from various projects business area wise, names of employees contributing highest change requests, nature of change requests (if there are too many change requests only asking for cosmetic changes like slight improvement to language, syntax etc, then perhaps not much thinking is 4happening). Too many Process Change Requests received could mean that the QMS processes are not stable or that they are defective.

3.3 Quality Planning—Some Interesting Dilemmas and Observations

It is well known that "quality" owes its origin to the manufacturing industry. Fine concepts like "Quality Circles"—Ref [4] etc. have their

roots in the manufacturing industry. The notion of "root cause analysis" finds its origins in the industrial engineering discipline.

Software industry is relatively young in implementing these practices in the pursuit of bringing the fine practices of classical engineering into the business of "software engineering". Quality is a buzz word today; to many its like a nice healthy child; nice to admire from a distance but difficult to take the pains of bringing it up with a myriad of thankless tasks.

Through the discussion so far, is should be clear that the Software Quality Assurance Plan is a vehicle to put the quality policy in practice. It is a mechanism whereby the procedures/processes defined in the Quality Management System get implemented. We also know that senior management commitment to quality gets expressed through the Quality Plan. In reality, however, there are many interesting practices noticed. . At the time of proposal writing, practitioners will come running to the quality manager and ask - "can you tell me where can I find a "nice" write-up available in our organization? We'd like to include it in our proposal to the client." At this time, probably, the business manager/practitioner knows it fully well that there is no go without including this write-up, that the proposal cannot really be completed without this "nice" write-up on organization's "commitment" to quality. It'd woe the customer; it would help him fetch the contract, the much-needed feather in his/her cap.

In chapter two, we discussed the various expectations that organization players would have from the Quality Management System. When the project gets initiated as per organization's process, the quality goals get documented; it's the same manager who starts finding the QMS a "big nuisance". Attempts after the attempts are made to undo the "leash of QMS processes". The tirade begins between the project SQA and the Project Manager (who believes that, his/her job is to push the product out of door or deliver the services to client as long as it works that way). After all, he can always look at process compliance at the "end of the project" or write a "nice" paragraph about it in the project wind-up project; that too because "quality people" insisted on it. It could even be that half of these project managers could be certified project managers who, as a part of their certification preparation, vouched for "quality practices"; Wasn't it one the domains they had to study for their exam! And once they become certified project managers, don't they help their organizations by getting their "rates" up?

Consider one more scenario, when once in a while the (say *quarterly* or *bi-annually*; whatever the defined frequency) the reporting to management begins, the "circus" really starts. A "symbiotic being" is discovered once again. QA department goes on a talent scoot to look for best practices, to look for assets that can be submitted etc.

Yet another scenario; when a customer-visit is announced, projects would become very friendly with the Quality Department. They want

the "impressive metrics" to be presented to the client to have the mesmerizing effect. The reality is that to collect periodic metrics from these same projects would probably be the worst nightmare for the quality team. Experience would not differ much if your were a quality consultant rather than a full time quality manager; at times your client's imagination about your "magic abilities" would range from expecting you to fudge the metrics to "creating" data on "actual efforts" for example.

In Chapter 1, we discussed about quality being a business issue. The reality is that having a "good quality department" for most Indian software organizations is like having a "smart enough" wife; she can cook well, do good English conversation in parties, enjoys a bit of drink with her husband, can dance a bit.. You name it and she can do it. And yet when the husband says, "shut up" she will be quiet until he allows her to speak again. With due consideration to the fact good exceptions do exist, it's a question worth an introspection to the Indian software industry-"do we dare to give enough authority to our quality department to put their foot down and stop the shipment of a product/services if it is not found to be worthy of release/shipment?" Are we capable of educating our clients right from the beginning that quality is a serious business issue and that we want to be married to it right from start to end of a project; focussed on long-term victory rather than being driven by some petty short-term goals? How many business managers would come to terms with this fact? Will the senior management get over the dilemma "how much quality" is the right amount of quality?" "How much process-suaveness is just enough and when to give it a back-seat?" At times, it is observed that even if the project manager is following all the processes 100% along with the readiness of his team members (and yet managing the project "successfully" from the client" point of view), his senior management would lament the fact that he/she is probably spending far too much time on "quality".

When a client tries to push a Change Request implementation "quickly" enough for us to be called a "co-operative" supplier; and you tell them that as per our QMS we need to get it through the Change Control Board, the client may frown. At this point of time, a question to ask ourselves is did we not "tame" our client on quality thinking right from the beginning? Did we give a realistic estimate to the client? Were we honest enough to tell our client that quality requires efforts? Or we took Phil Crosby so seriously that we created an impression to our client's mind that "quality is free", "effortless" i.e. that we're naïve enough to believe that commitment to quality comes without putting in any hard efforts"?

Of course, there is the other side of the coin too. Quality professionals need to have the "tact" of "selling" the QMS processes to the practitioners to see its realistic and yet meaningful implementation. Better, if they come from Project background; having burnt their

fingers on "misguided estimates", having lost their hair in managing customer expectations. If a quality professional comes from project implementation background rather than either an academic or theoretical background, then there would a far more conviction in his/her appeal to practitioners to follow the QMS processes. That is what we meant in the section 2.4.8 of Chapter 2 on "People issues in Software Quality Assurance.

REFERENCES

[1] *IEEE Guide to Software Requirements Speciation —ANSI/IEEE Std 830*
[2] *Software Engineering: A Practitioner's Approach*—Roger S. Pressman (McGraw-Hill International)
[3] *The Software Factory: Managing Software Development and Maintenance*—James R. Johnson (BPB Publications)
[4] *The Essence of Total Quality Management*—John Bank (Prentice Hall India)

Chapter 4

PRODUCT QUALITY AND PROCESS QUALITY

4.1 Introduction

In the first three chapters, we discussed about the business case for Software Quality Assurance, the role of the SQA function and how it should be organized, whom it should report to. Personality traits required in a Software Quality Analyst to succeed in his/her job in SQA function were also discussed. We further discussed how the quality managmemet system (QMS) gets deployed in the organization and the various expectations from it. We also discussed the typical items that are under SQA review. Our discussion in Chapter 3 was focussed on how to plan for Software Quality Assurance, typical contents of an Software Quality Assurance Plan etc.

With this background created, in this chapter we'll discuss two important concepts—*"Product Quality"* and *"Process Quality"*. This will be done with the help of various models for software product quality and process quality. It is important to distinguish between PRODUCT QUALITY and PROCESS QUALITY because it has a bearing on how an organization in software business wants to organize its mechanism to reach the ultimate objective in gaining an edge in through quality. Although discussion of product quality and process quality is akin to the discussion of "measures and metrics", we have chosen to take it up in a separate chapter.

Software Engineering Institute (famously known as SEI) at Carnegie Melon University has done extensive work in this area known as *"Process Improvement"*. It is a site well worth visiting (www.sei.cmu.edu). SEI also distinguishes between "product metrics" and "process metrics"; this is discussed in the chapter on "software metrics" (classification of software metrics). IEEE (Institute of Electrical and Electronic Engineers) also has an extensive number of standards (www.ieee.org). We'll start this chapter with a discussion on "Product Quality". We'll discuss

"Process Quality" next. The issue of "Process Quality" will be approached from the "maturity" viewpoint and at the end we'll discuss the relation between them. In addition to this, a separate chapter is devoted to provide an overview to SEI's Capability Maturity Model, which is basically a well-known framework for *continuous process improvement*. (Process Improvement frameworks are discussed in detail in a chapter devoted to the topic).

4.2 Software Systems Evolution

Much has happened since in the last 40 or so years of computer invention. If we look back, we realize that each era was characterized by focus on certain kinds of software systems. Table 1 below summarizes this.

Table 1: Software Systems Evolution

Name of the Era and the Period	*Characteristics of the systems*
The Early Era (1950—60's)	• Batch processing oriented systems • Limited Distribution •Customized Software
The Second Era (1960—70's)	• Multi-user Systems • Real Time Systems • Databases • Product Software
The Third Era (Mid 70's—Late 80's	• Distributed Systems • Embedded Software • Low cost Hardware • Consumer Impact
The Fourth Era (Late 80's to just before Millennium)	• Powerful Desktop Systems • Object Oriented Technologies • Expert Systems • Artificial Systems and Neural Networks • Parallel Computing • Network Computing
Current Era	• Pervasive Computing • Internet based systems • Growth in Intranets • E-Commerce • M-Commerce (mobile e-Commerce) and Wireless Computing

In each era, the problems and challenges faced by the producer software product have not changed much: (Interested readers are directed to ref [1] for an excellent discussion on this.)

- Why does it take so long to get the software product/project finished
- Why are software development costs so high

- Why can't we find all errors before we deliver the software to our customers
- Why do we face difficulties in measuring the progress of software product/service development

Software products, as compared to non-software products have certain characteristics:

- Software is "developed" or "engineered"; it is not "manufactured" in the classical sense
- Software does not "wear out"; it only deteriorates over a period of use—it needs "Enhancements"
- Most software today is assembled from re-usable components rather than being custom-built based on fresh requirements analysis

4.3 Product Quality

If we reflect on these points and look around for our own experiences during software product development, we realize that it is the project infrastructure, which provides the framework through which software quality is "built in" the software product. The framework is really the "process" aspect of software product development. The "attributes" and characteristics of the software products and the degree that they fulfill specific project needs provides an incremental measure of quality of the end product. Product attributes are defined, first, as attributes for each data product produced, as the software is developed and second, as attributes that must be integral to the software when delivered for system integration. Before moving on with our discussion on "product quality" let's consider some basic definitions (as in IEEE Std. 610.12 IEEE Standard Glossary of Software Engineering Technology). It defines *"Software Product" as (1) The complete set of computer programs, procedures, and possibly associated documentation and data designated for delivery to a user. (2) Any of the individual items in (1)*

4.3.1 Software Attributes

According to IEEE STD 983-1986, "Standards" are *Mandatory requirements employed and enforced to prescribe a disciplined uniform approach to software development, that is, mandatory conventions and practices are in fact standards.* In order to ensure that development product meet a defined quality standard when delivered, standards and practices must be defined early. They must be specific to the software products. There must be quality "gates" identified, which will monitor the quality of the software products being produced before they are used by the project. Without a means to measure conformance to standards, prac-

tices and conventions, quality of a software product cannot be monitored.

Standards, Practices and Conventions

As said above, IEEE standard 610.12 defines "Standard" to be Mandatory requirements employed and enforced to prescribe a disciplined uniform approach to software development, that is, mandatory conventions and practices are in fact standards.

Quality Assurance Institute (QAI) says that "**Standards**" are the measures used to evaluate products and identify nonconformance; a basis upon which adherence to policies is measured.

IEEE standard 610.12 defines "**Practices**" as Requirements employed to prescribe a disciplined uniform approach to the software development process.

Conventions (according to IEEE Std. 983) are requirements employed to prescribe a disciplined uniform approach to providing consistency in a software product, that is, uniform patterns or forms for arranging data.

"Quality" is really a composite of many characteristics—it is best to capture the notion of quality in a model that depicts these composite characteristics and their interrelationships. Sometimes, the distinction between "internal" and "external" attributes of software product becomes blur, making it difficult to understand exactly what software product quality is! Still, *Quality Models* are important because they tell us what people think is important and also help us to understand the commonalities of their views. In this section, we want to look at some very general models of software product quality that have gained acceptance within the software engineering community.

Several researchers have decomposed the notion of "*Software Product Quality*" into a number of features or characteristics. Such features typically include:

- **Reliability:** The ability of a system or component to perform its required functions under stated conditions for a specified period of time.
- **Usability:** A set of attributes that bear on the capability of software to maintain its level of performance under stated conditions for a stated period of time.
- **Functionality:** A set of attributes that bear on the existence of a set of functions and their specified properties. The functions are those that satisfy stated or implied needs.
- **Maintenability:** A set of attributes that bear on the effort needed to make specified modifications.

- **Correctness:** (1) The degree to which a system or component is free from faults in its specification, design, and implementation. (2) The degree to which software, documentation, or other items meet specified requirements. (3) The degree to which software, documentation, or other items meet user needs and expectations, whether specified or not.
- **Portability:** A set of attributes that bear on the ability of the software to be transferred from one operating environment to another.
- **Testability:** The effort required to test a program to ensure that if performs its intended functions.
- **Efficiency:** A set of attributes that bear on the effort needed to make specified modifications.

Perhaps, McCall, in the Factor-Criteria-Metric, gave the first definition of the characteristics of *software product quality*. Basili and Rombach and Boehm have done considerable work in the area of proposing models for Software Product Quality Ref [6]. A detailed discussion on this is taken up in the following sections.

4.3.2 Classification of Software Application Attributes

Specific attributes for software products and services are defined based on the requirements of the project. These attributes are specific to the application and fall into several categories:

1. *Performance Attributes*—attributes, which describe the execution characteristics of the software when, integrated into an operational configuration. (*Configuration is the arrangement of a computer system or components defined by the number, nature and interconnections of its constituent parts*)
2. *Form Attributes*—Those, which describe the form of the product and how it will appear when delivered (the look and feel). This is important from the view point of the end user
3. *Processing Attributes*—Those which describe processing characteristics of the software
4. *Functional Attributes*—Those, which describe the functionality of the system when, integrated into an operational configuration. (The chapter on TESTING will discuss various aspects of functional testing.)
5. *Operational Integrity*—Those, which describe the reliability, system control, and operational support characteristics of the software and the degree that the system supports the requirements of the application. (*"Reliability" is the ability of a system of component to perform its required functions under stated conditions for a specific period of time)*

6. ***Maintainability***—Those which describe the ease with which the software may be updated to reflect changing system or operational needs to correct operational discrepancies. (In the chapter on Software Measurements and Metrics, measurement aspects relating to Maintenance are discussed)

4.4 Models for Software Product Quality

4.4.1 McCall's Factor-Criteria-Metric Model

This model assumes that there are a number of high-level quality factors of any software product (e.g. *usability*) and that these factors may be defined in terms of criteria (e.g. *modularity*). At the lowest level are the *metrics* (more on this in the chapter "Software Measurement and Metrics"), which indicate or measure the criteria. The purpose of McCall product quality model was to define the characteristics to be assessed and to suggest some measures to capture those characteristics. McCall has described quality using a decomposition approach, which is depicted in Table 2. Figure 1 presents Boehm's view of software product quality.

In models such as these, (McCall and Boehm) the focus is on the final product (usually executable code) and they identify key attributes of quality from the user's perspective. These key attributes, called *quality factors* are normally high-level external attributes like "reliability", "usability", and "maintainability". But they may also include several internal attributes; such as "testability" and "efficiency". Each of the models depicted above assumes that quality factors are still at too high

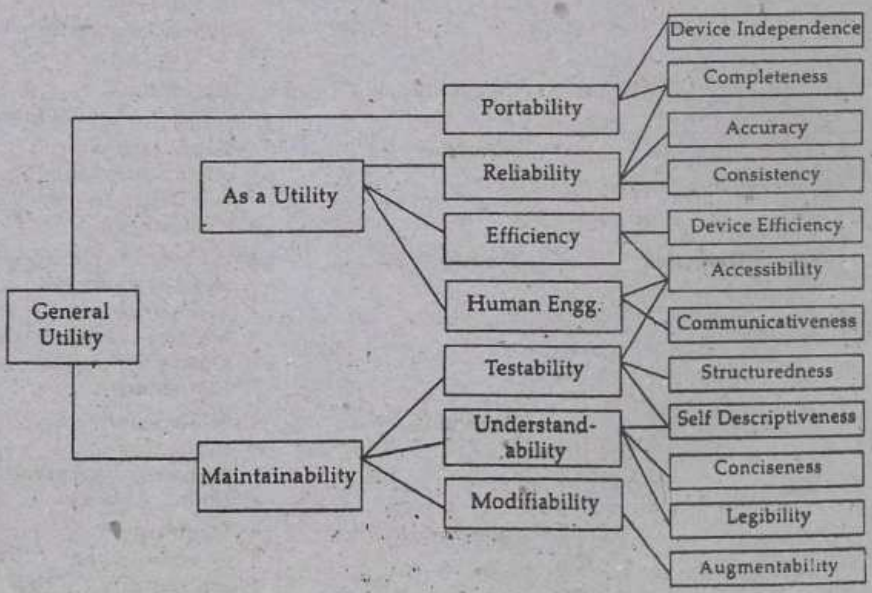

Figure 1: Boehm's Software Product Quality Model

a level to be meaningful or to be measurable directly. (Selection of the right kind of measures for various process and product attributes is discussed in the chapter on Software Measurements and Metrics.) When these high-level quality factors are decomposed into lower level attributes, we get *quality criteria*. The McCall model was originally developed for the US Air Force and its use is promoted within the US Department of Defense for evaluating software product quality.

Table 2: McCall's Software Product Quality Model

Use	*Factor*	*Criteria*
Product Operation	• Usability	• Operability • Training • Communicativeness • I/O Volume • I/O Rate
	• Integrity	• Access Control • Access Audit
	• Efficiency	• Storage requirement • Execution speed
	• Correctness	• Traceability • Completeness • Consistency
	• Reliability	• Accuracy • Error Tolerance • Consistency • Simplicity
Product Revision	• Maintainability	• Consistency • Simplicity • Conciseness • Self-descriptiveness • Modularity
	• Testability	• Simplicity • Scope for Automation
	• Flexibility	• Expandability • Architecture Usability • Self Descriptiveness • Modularity
Product Transition	• Re-usability	• Architecture Usability • Self descriptiveness • Modularity • Machine Independence • Operating Systems Independence
	• Portability	• Self descriptiveness • Modularity • Machine Independence • OS Independence
	• Inter-operability	• Modularity • Communications commonality • Data commonality

4.4.2 The ISO 9126 Standard Quality Model

For a long time, the user community was looking for a single model for depicting and expressing quality. A universal model makes it easier to compare one product with another. In 1992, a derivation of the McCall model was proposed as the basis for an international standard for software quality measurement. This is called *"Software Product Evaluation: Quality Characteristics and Guidelines for their Use"*. This standard is more commonly referenced by its assigned standard number, ISO 9126 (1991). As per this standard, software quality is defined to be: *" The totality of features and characteristics of a software product that bear on it ability to satisfy stated or implied needs"*. Quality is decomposed into six factors:

- Functionality
- Reliability
- Efficiency
- Usability
- Maintainability
- Portability

The standard claims that these six attributes of product quality are comprehensive; i.e. any component of software quality can be described in terms of some aspect of one or more of the six factors. In turn, each of the six is defined as a "set of attributes that bear" on a relevant aspect of software, and each can be refined through multiple levels of sub-characteristics.

4.4.3 Other Models for Software Product Quality

Goal-Question-Metric Model is another well-known model from Basili and Rombach. The other one is by Gilb; we'll discuss this a little later. The basic idea behind the work of Basili and Rombach is that metrics are derived from measurement goals (for which an organization committed to quality must have a well defined processes for quality planning and measurement) and questions, usually related to quality characteristics.

Gilb (and Kitchenham) have pioneered the "define-your-own-model approach. This method can be thought of as "design by measurable objectives"; it complements his philosophy of *evolutionary development* (See the vignette below on *Evolutionary Models*) of the software product. Gils's approach is similar to work by others including McCall, in that high-level features of software product quality are decomposed into more measurable attributes. This is the philosophy underlying in Gilb's model/approach to software quality; the software engineer delivers the product incrementally to the user or market, based on the importance of the different kinds of functionality being provided. To

assign priorities to the functions, the user identifies key software attributes in the product specification. These attributes are described in measurable terms, so the user can determine whether measurable objectives (in addition to the functional objectives) have been met. Figure 2 illustrates and example of this approach.

The COQUAMO (Constructive QUAlity Model) approach of Kitechenham and Walker extends Gilb's ideas and supports them with automated tools as part of an ESPRIT (European Community) project.

Evolutionary Software Models

There is a growing recognition that software systems typically are complex and as such they evolve over a period of time. Business product requirements often change as development proceeds. This makes it difficult to follow the linear sequential approach to software development. To accommodate the ever changing market requirements, tight market deadlines, a one shot release of software product is impossible. Instead, a limited version must be introduced to meet competitive or business pressure. In such cases, it becomes essential to adopt an "evolutionary approach" to software product development. Evolutionary models are iterative. They enable software engineers to develop increasingly more complex versions of the software product.

An important point to be noted that all product quality attributes may not go together; in fact some can be in direct conflict with others. This is illustrated in Table 3. The person eliciting software requirements must keep this in mind.

There is a significant amount of ongoing effort within the software industry, academia and government, directed towards developing the entire area of what is commonly called "Software Quality Metrics" - that is how to describe the overall characteristics of software, namely attributes, and the measurement of those attributes, namely metrics.

IEEE defines Quality Metric as (1) A quantitative measure of the degree to which an item possesses a given quality attribute. (2) A function whose inputs are software data and whose output is a single numerical value that can be interpreted as he degree to which the software possesses a given quality attribute.

4.5 Process Quality

A commonly used definition of software quality as defined in ISO 8402 international standard is:

Quality is the totality of features and characteristics of a product or service that bear on its ability to satisfy stated or implied needs.

Table 3: Relation among Software Attributes

Factors \ Efficiency	Flexibility	Integrity	Inter-Operability	Maintain-ability	Port-ability	Reliab-ility
Efficiency						
Flexibility	X					
Integrity	X					
Inter-Operability	X	O	X			
Maintain-ability	X	O				
Portability	X			O		
Reliability	X			O		

X Factors Conflicts O Factor Supports Blank No Relation

(For a complete list of software factors, readers can refer to IEEE STD 1061)

Flexibility *The ease with which a software system or component can be modified for use in applications or environments other than those for which it was specifically designed*

Interoperability *The degree to which software can be connected easily to other systems in order to operate it*

Portability *The effort required to transfer the software code from one hardware and/or software system environment to another*

Given this definition there would appear to be no such thing as "process quality"; although by implication a quality process is one, which leads to the production of a quality product. To evaluate the process it has to be made tangible. One way of doing this is to base the process on a standard or model against which conformance may be assessed. (In Europe, the prevalent approach is to base the process and any evaluation of it on a standard (ISO 9001 or similar) while in the United States the focus for process quality improvement and evaluation is a process 'maturity model'. (A detailed discussion on standard models will be taken up in chapters *Software Certifications and CMM Mode*)

But both approaches are discussed in brief in this section, bearing in mind that process evaluation only takes account of and is ultimately only as good as, the underlying standard or model used. The rationale is : A Process Approach implies that every activity in an organization has a defined start and finish and can be measured. (The topic of *Measurements and Metrics* is discussed in detailed in a separate chapter.)

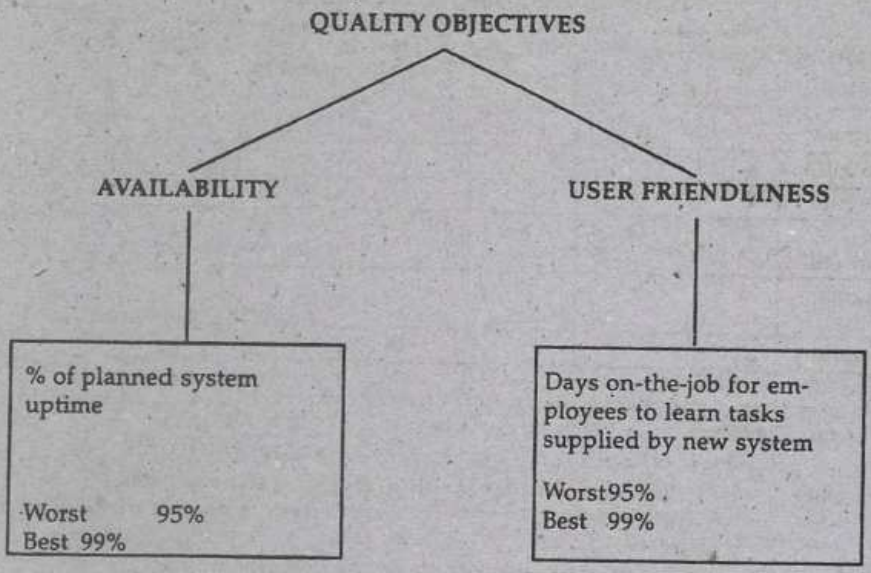

Figure 2: Gilb's Approach to Software Product Quality Model

4.5.1 ISO 9001 Quality Management for Process Quality Framework

One approach to the achievement of high quality is based on the use of an "adequate" predefined production process. At best, Quality Management Standards provide a baseline for an adequate production process by subjecting the processes to fulfill certain key requirements. In doing so, the attempt is to ensure the capability and commitment of organizations to produce quality products. The leading international Quality Management Standard (QMS) for software development is the International Standards Organization's generic 'Quality Systems' series of standards, ISO 9000 to 9004. When we refer to the ISO 9000 series, we will normally mean ISO 9001, which has been tailored for use in software environments by the addition of software-specific guidance documentation. ISO 9001 makes reference to "the process approach" to managing an organization. Figure 3 provides a graphic representation of what is famously known as the PDCA cycle (PLAN, DO, CHECK, ACT). In applying ISO 9001 to software it has to be recognized that software differs in a number of ways from other industrial products, and that the processes used to produce software are not typical industrial processes.

International Standard ISO 9000-3, *Guidelines for the Application of ISO 9001 to the Development, Supply and Maintenance of Software*, helps address some of these differences with references to the software lifecycle and supporting activities. The general framework of ISO 9000-3 includes sub-sections on management, quality system, internal audits and corrective actions. The standard assumes a lifecycle model of some

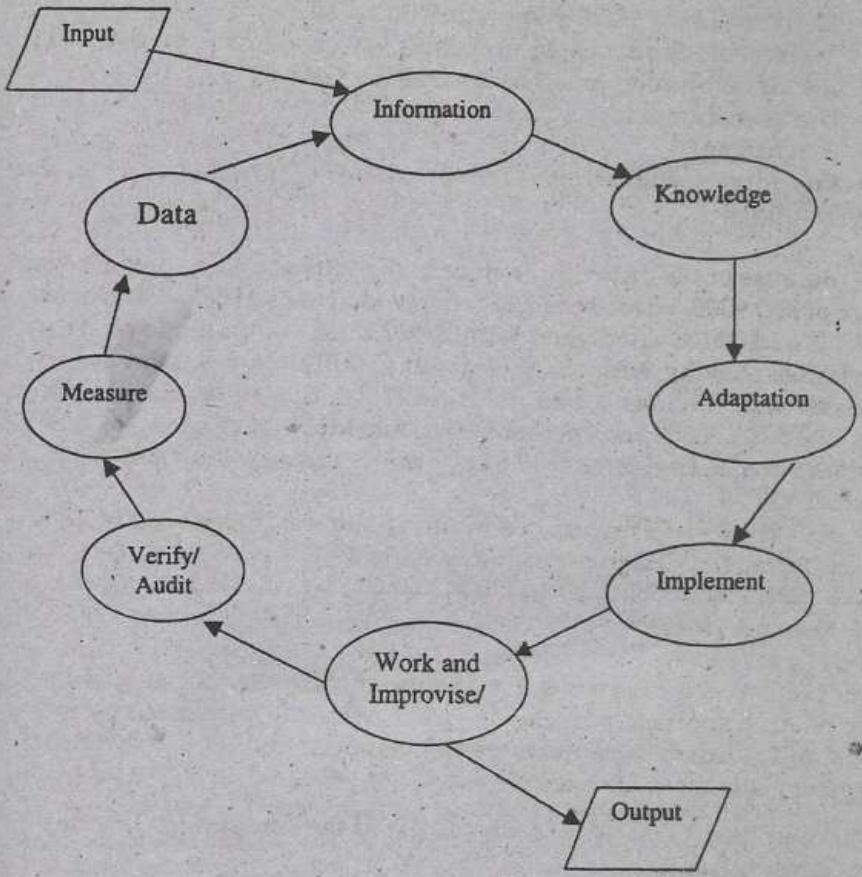

Figure 3: The Process Approach

type is used but does not prescribe any particular one. References to requirements, design, implementation and testing all appear in the standard. A number of plans are required by the standard, including:

- A development plan
- A quality plan
- A test plan
- A maintenance plan

A proposed structure of each of these plans is itemized. The emphasis throughout lifecycle selection is on the definition and control of the software process in accordance with established good practices in software production. Supporting activities in the ISO 9000-3 standard include:

- Configuration Management (there is a separate chapter about this)
- Document Control

- Maintenance of Quality Records
- Process and Product Measurement
- Adoption of rules, practices and conventions
- Use of tools and techniques
- Purchasing
- Procedures for included software
- Training

Because of the distinctive nature of the software process, the structure of ISO 9000-3 does not strictly follow that of ISO 9001. Much of the ISO 9001:2000 is concerned with describing the quality system in 5 clauses starting with clause 4 about Quality Management System (these will be described in details in the separate chapter on ISO 9001:2000) and their sub-sections. Most, but not all, of these subsections have given rise to one or more corresponding subsections in ISO 9000-3.

ISO 9000-3 was reproduced in full by the TickIT project. EN 29001 is the European designation for ISO 9001.

A Note on TickIT

TickIT is about improving the quality of software and its application. An important purpose of TickIT, which is supported by the UK and Swedish software industries, has been to stimulate software system developers to think about:

* What quality really is in the context of the *processes* for software development?
* How quality may be achieved, and
* How quality management systems may be continuously improved

4.5.1.1 Role of ISO 9001 in Evaluation of a software product

ISO 9001 was designed for use between two parties; the *purchaser* and the *supplier* involved in a contract for the development of a particular (software) product. However, the standard is now also used for independent assessment of a (software) *production process*, a purpose for which it was not originally intended. It should be noted, however, that QUALITY MANGEMENT SYSTEMs such as ISO 9001 are little more than very general guidelines, which help to ensure that all reasonable steps have been taken to encourage *process quality*. Although undoubtedly of value, there is little ground for supposing that their use inevitably leads to the manufacture of quality products. Among the reasons for this one is that quality management systems do not rigidly define the *development process* and their successful implementation is dependent on strict adherence and good workmanship (for software products, this would mean programming practices, coding guidelines,

GUI standards etc). For evaluating product quality through certification, an impact could be dependent on availability of widely recognized *process certificates*. For example, the availability of a comparatively cheap process certificate covering a number of software products may disincline organizations to opt for product certification for every product. Conversely, but to a lesser extent, the availability of such certificates may reduce certification costs by limiting the overhead of certifying different releases of the same product where it is felt that a re-evaluation is not necessary.

4.5.2 *Maturity Models for Process Quality*

We have a separate chapter dealing with the CMM (Capability Maturity Models). In this section we wish to provide only a brief overview of the CMM. For details, readers are directed to www.sei.cmu.edu.

The Software Engineering Institute (SEI), a federally funded research and development laboratory based at Carnegie Mellon University in the United States, has developed a model of the software development process known as the "Capability Maturity Model (CMM)". The model is used as the basis for *process improvement* and evaluation. Many documents relating to the model are kept in the public domain and are available for downloading at the site mentioned above.

The CMM is based on the quality principles first espoused by Walter Shewart in the 1930s and first cast into a 'maturity framework' by Philip Crosby (1979) [Ref 8]. Crosby discusses fourteen principles for a *quality improvement programme*. Shewarts principles were further refined and demonstrated during the 1980's by, among others, Edwards Deming (1982) , Juran (1988, 19889) and Ron Radice *et al.* (1985). At the time, Radice and colleagues were working at IBM under the direction of Watts Humphrey. Humphrey brought the concepts to the SEI and made this the foundation of the SEI's process maturity framework, with a view to assessing the software engineering capabilities of American Department of Defense (DoD) contractors.

Basically, the SEI framework identifies five 'maturity levels' (initial, repeatable, defined, managed and optimizing), on an ordinal scale (for discussion on various types of scales, readers can refer the chapter on metrics and measurements) which roughly corresponds to a progression of candidate (software) process from 'ad hoc' (unpredictable, poorly managed), through intuitive (basic project management practices and the ability to repeat previously mastered tasks), qualitative (well defined and institutionalized), quantitative (where the process is 'measured' and 'controlled'), up to the point where a process is in a continuous state of optimization based on measurement feedback.

The basis premise in this process improvement model is that if the quality of process for developing for a software product is good then there is a reasonable assurance that the quality of the end product is also likely to be good ('quality' as discussed in various sections of this book). For convenience, the practices that describe *key process areas* are grouped into the following *'common features'*:

- Commitment to perform
- Ability to perform
- Activities performed
- Measurement and Analysis
- Verifying implementation

It may be noted that since the CMM, unlike ISO 9001, is software specific, there is not need for guidance in relating the model to software. SEI offers various training courses (mainly for assessors and for general awareness); the details can be found at the SEI mentioned earlier.

Conclusions

In this chapter, our aim was to give insights into 'product quality' vis-à-vis 'process quality' as this is the foundation for discussion software metrics and measurement. If an assessment of the (software) development process is to be a factor in evaluation and certification of a software product, it will be essential to establish some quantitative criteria as to the value of the process in relation to the quality of the product. To the best of author's knowledge, no such direct relationship exists—it is simply assumed that a well managed and disciplined (software) development process leads to a superior quality product. However, practical experience in the industry show that a 'good' process in itself is no guarantee of quality, especially if the process itself does not enforce good workmanship or adherence to well well-defined software development standards; coding guidelines, GUI standards etc. On the other hand, it is also true that a bad process is likely to engender good product quality; for example, lack of well defined and practiced process for peer reviews and inspection.

REFERENCES

[1] *After the Gold Rush*—Steve McConnell (Microsoft Press)
[2] *The Mythical Man-Month*—Frederick Brooks (Pearson Education)
[3] *Applied Software Measurement*—Capers Jones (McGraw Hill)
[4] *Software Metrics—A Rigorous & Practical Approach*—Norman Fenton and Shari Pfleeger (Thompson Books)
[5] *Software Quality Assurance & Management* —Michael Evans and John Marciniak (John Wiley and Sons)

[6] *Software Evaluation for Certification—Principles, Practice and Legal Liability*—Andrew Rae, Philippe Robert, Hans-Ludwig Hausen (McGraw Hill)
[7] *Microsoft Secrets*—Michael Cusumano and Richard Selby (Harper Collins)
[8] *Quality is Free*—Philip B. Crosby
[9] http://www.humanfactors.com/downloads/guistandards.pdf
[10] *The Goal Question Metric Approach*—Victor R. Basili 1 Gianluigi Caldiera 1 H. Dieter Rombach (Institute for Advanced Computer Studies, Department of Computer Science University Of Maryland—USA)

Chapter 5

SOFTWARE MEASURMENT AND METRICS

5.1 Overview

Some twenty to twenty-five hear ago, Software Metrics was a curiosity confined to a few university researchers and one or two industrial or commercial organizations. Today, Measurement is becoming increasingly accepted part of software engineering, but it is not always a well-understood one. Also, one of the problems facing the information technology (IT) industry today is the application of measurement-based techniques in a business environment. The world seems to be dichotomously divided between "measurement believers" and non-believers! It is said that we see what we want to see. In the software world, it is better to change this attitude to—"we'll see only if we measure".

A key element of any engineering process is "measurement" Measures help us to better understand the attributes of the software products or systems created and to assess the quality of those engineered products/systems. Measurement is the process by which numbers or symbols are assigned to the attributes of entities in the real world in such a way as to define them according to clearly defined rules.

While doing this, a distinction is to be made between "measures" as representations of observable attributes of objects or processes and prediction systems that combine measurements and parameters in order to make "quantitative predictions". This is an important distinction when validating measures. By recourse to measurement theory (a discussion on this out of the scope of this book), it can be determined whether the measure is proper in the sense of representing the attribute adequately. However, empirical validation of the engineering usefulness of a measure requires an explicit prediction system. Also, it is important to distinguish between the terms "measure" and "metrics"; whereas a measure is a pure number, metrics is most typically a ratio of two or more related measures. Figure 1 below illustrates this point about the difference between measurement and metric.

Figure 1: Definitions

NUMBER	– an 'un-described' value ("5" is a NUMBER)
DIMENSION	– a descriptor "defects" is a DIMENSION
MEASURE(ment)	– a NUMBER with a DIMENSION "5 defects" is a MEASURE
METRIC	– two related MEASURES "6 Defects / KLOC is a METRIC
UNIT	– a target or area of measurement

In this chapter, we have covered:

- What is measurement and the need for measurement
- Role measurement in software development life cycle
- Defect Metrics
- Metrics for software maintenance
- Software metrics classification
- Requirements related metrics in view of Integrated CMM (CMM-I)
- Role of Metrics in process improvement
- Measurement principles
- Selecting appropriate metrics for projects and benefits of metrics for proactive project tracking
- Metrics Implementation
- Earned Value Analysis
- Planning for metrics program in an organization
- Metrics Program Implementation Issues
- Object Oriented Metrics

The next section will discuss each of above points briefly. We'll also provide a classification of Software Metrics. Toward the end of the chapter some planning issues are discussed along with the issues faced in reality while implementing a metrics program in an organization.

5.2 Introduction

5.2.1 What is Measurement?

This chapter is about measurements and metrics. It is therefore appropriate to do a brief discussion on what we mean by measurement. As said above, by its nature, engineering is a quantitative discipline. Engineers use numbers to help them design and assess the product to be built. It is essential that the same approach is adopted while designing software products. Also, measurement is such an everyday activity that we take it for granted. Measurement is defined as:

The process of assigning symbols, usually numbers, to represent an attribute of the entity of interest, by rule.

A definition of Software Metrics is [4]:

The continuous application of measurement-based techniques to the software development process and its products to supply meaningful and timely management information, together with the use of those techniques to improve that process and its products.

There are a number of points to note from this definition. First, the entity that we wish to measure can either be an object (for example, a person or a computer program) or it could be a process (for example, traveling from place A to place B or say debugging some code). Second, there must be some distinct attribute that we wish to measure; examples are: height, length, duration and cost. Third, typically, although not necessarily, we use numbers to represent the attribute being measured—for example length of a computer program could be represented by 315. Lastly, it must be noted that there is the requirement that the assignment of numbers, or whatever, must be according to some explicit rule.

Often, you may hear a term "indicator" in the literature on metrics. The term indicator is used to denote a representation of metric data that provides insight into an ongoing software development project or process improvement activity. Indicators are metrics in a form suitable for assessing project behavior or process improvement. For example, an indicator may be the behavior of a metric over time or the ratio of two metrics. Indicators may include the comparison of actual values versus the plan, project stability metrics, or quality metrics. Examples of indicators used on a project include actual versus planned task completions, actual versus planned staffing, number of trouble reports written and resolved over time, and number of requirements changes over time.

5.2.2 *Why Measure?*

We measure in order to answer questions—How large?, How maintenable? Does technique 'A' yield more productivity gains than technique 'B'? Will 'this' approach be more cost-effective than 'that'? What characterizes error-prone programs? How long will it take? Thus, if we have no questions, we have no need for measurement. Measurements inform our analysis and hone our critical faculties; it helps us to overcome subjectivity and provide a level of precision not otherwise possible. This does not mean that measurement is a panacea—misuse is all too possible—however, it has the potential to underpin computing as an observational science and software development as an engineering activity. From this, it should be possible to appreciate that the potential diversity of software engineering related measures, generally termed "software metrics" is vast. (Typically, whereas measure is a pure number as discussed above, metric is usually a ration of two or more

measures—for example 30 defects v/s 30 defects per 1000 lines of code, or as another example—50 kilometers v/s 50 kilometers per hour)

Let us now discuss some examples for application of measurement. One example is to monitor software reliability over a period of time. Is it improving? If so, what is the rate? Another example could be to assess the maintainability of aging software systems and to direct re-engineering or re-structuring activities. As yet another example, consider software engineers applying measurement for early identification of a project going off the track. The idea of measurement as part of software development is not new. Right from the pioneering days of computers, people have been concerned with measurement related questions.

5.2.3 What are the Steps in Measurement?

Step 1

The first step in the measurement process is to identify the software measures and metrics that are "appropriate" for the representation of software that is being considered. Next, data required to derive the formulated metrics are collected. Once computed, appropriate metrics are analyzed based on a pre-established guidelines and data on past performance. The vignette on the next page illustrates how a project can make use of past data to set quality goals. Figure 2 shows the Control Chart for the same data based on hypothetical data on the acceptance test defects. For an excellent discussion on control charts and other analytical techniques for software measurement and metrics, reader is strongly urged to see [13] from the reference list at the end of this chapter.

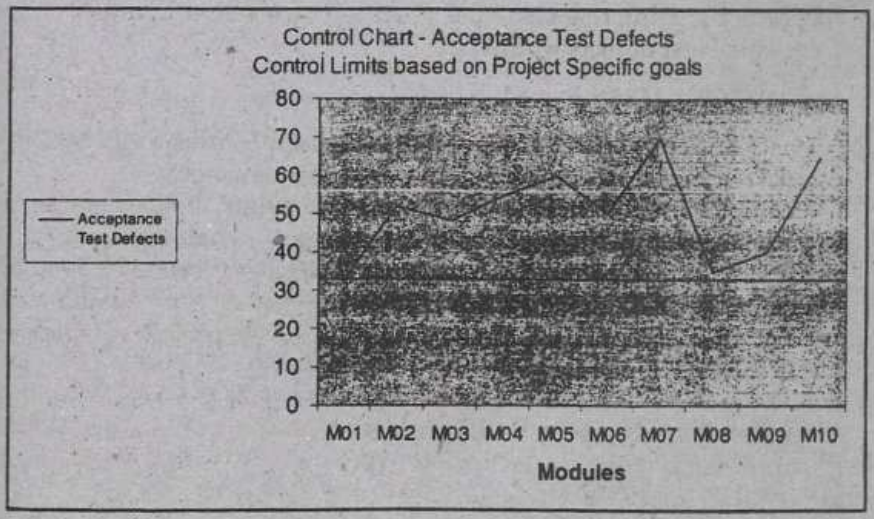

Figure 2: Control chart

Use of Past Performance Data: An Illustrative Example

Suppose 'Yojana' (Phase 2) is a project that involves two system components developed earlier, each handled as an independent project. Suppose these three projects (Yojana, Component 1 Project and Component 2 Project) are all similar in many respects (i.e. in terms of customer, development language and platform, end use domain, nature of the application etc.) Because of this similarity, the past data from two completed phases (development of Component 1 and Component 2) will be the best source of information for planning. Hence the data from earlier phases of Yojana (Component 1 and 2) is used. The Table below shows the acceptance test defects and total effort for the previous two projects.

Earlier Project	Effort Peron-Days	Defect detected during Acceptance Test	Integration/System Testing Defects
Component 1	528	17	54
Component 2	282	16	25
Total	810	33	79

Suppose, it is estimated that the effort for the current project (Yojana) is 1,420 person-days. Using *effort* as the basic predictor for defect levels, we obtain the estimated number of defects in *acceptance testing* for Yojana:

$$33 \times [1{,}420/810] = 58 \text{ defects}$$

Suppose the past data from *process capability baseline* shows 0.0030 defect per person-hour and if we were to estimate acceptance test defects by using this past data, then our estimate of number of acceptance test defects would be:

$$0.0030 \times 1420 \times 8 = 34 \text{ defects}$$

where an assumption of 8 Hrs per day is used (remember that in the table above the efforts are shown in person-day unit)
This value is lower than what is predicted by the past data on similar projects. The quality goal of this project is then set as a range rather than a fixed number. Thus the *quality goal* becomes *34 to 58 acceptance defects*. Figure 2 shows how the control chart would look for some hypothetical data from modules of the project with upper and lower control limits.

Step 2

As a next step, the results of the analysis are interpreted to gain insight into the quality of the software, and the results of the interpretation lead to modification of work products arising out of analysis, design, code or test. This is how the measurement program ties to the SDLC (Software/System Development Life Cycle). This aspect is discussed in section 5.3. As will be seen later, Figure 6 translates these steps in a diagrammatic form.

5.2.4 Attributes of Effective Software Metrics

Extensive work has been done in the area of software metrics. A wide variety of information sources on technical metrics and related subjects is available on the Internet [8]. Several hundreds of metrics have been proposed for computer software but all may not provide practical support to the software engineer. It is important to understand that metrics need to be innovatively designed to suit demands of each software project situation. In principle, the derived metrics and the measures should be:

- **An effective mechanism to enable high quality feedback:** the metric should provide a software engineer with information that can lead to a higher quality product.
- **Independent of the Programming language:** Metrics should be based on the phases of the life cycle model (Analysis, Design or structure of the program itself) but not dependent on the idiosyncracies of programming language constructs, syntax or semantics.
- Simple and computable: Simplicity is the essence of things! It should be relatively easy to learn how to derive the metric, and its computation should not demand inordinate effort or time.
- **Consistent and objective:** The metric should always yield results that are unambiguous. An independent third party should be able to derive the same metric value using the same information about the software.
- **Consistent in its use of units and dimensions:** The mathematical computation of the metric should use measures that do not lead to weird combinations of units (units of measure). For example, multiplying the number of people on the project teams by programming language variables in the program results in a spurious mix of units!
- **Empirically and intuitively persuasive:** This is a little esoteric but it is considered an important aspect—the metric should satisfy the software engineer's intuitive notions about the product or process attribute under consideration. For example, when schedule re-estimation done, depending on the past performance data, the band of allowed variance could tighten as compared to that with the first

estimation. This is because in the second attempt, one is expected to be in a better position to plan the schedule, as at this stage, more details are known about the project. For a detailed discussion on process variation etc reader should refer to [4] in the list of references listed at the end of this chapter

5.3 Measurement during Software Life Cycle Context

A project management measurement system is based on SCHEDULE, COST AND EFFORTS. Such a system adds value to all of the major phases of software life cycle. Figure 3 illustrates the major kinds of *measurements* associated with each phase starting from project initiation right up to the maintenance phase. Project accounting and configuration management shown in the figure are the umbrella activities in a software project in that, they encompass the entire project life cycle.

5.3.1 Measurement for Enhancement Phase

For projects that are *enhancements* (projects where the work involves extension of the existing functionality or the modules or components) or replacements or re-engineering of existing systems, the *structure,*

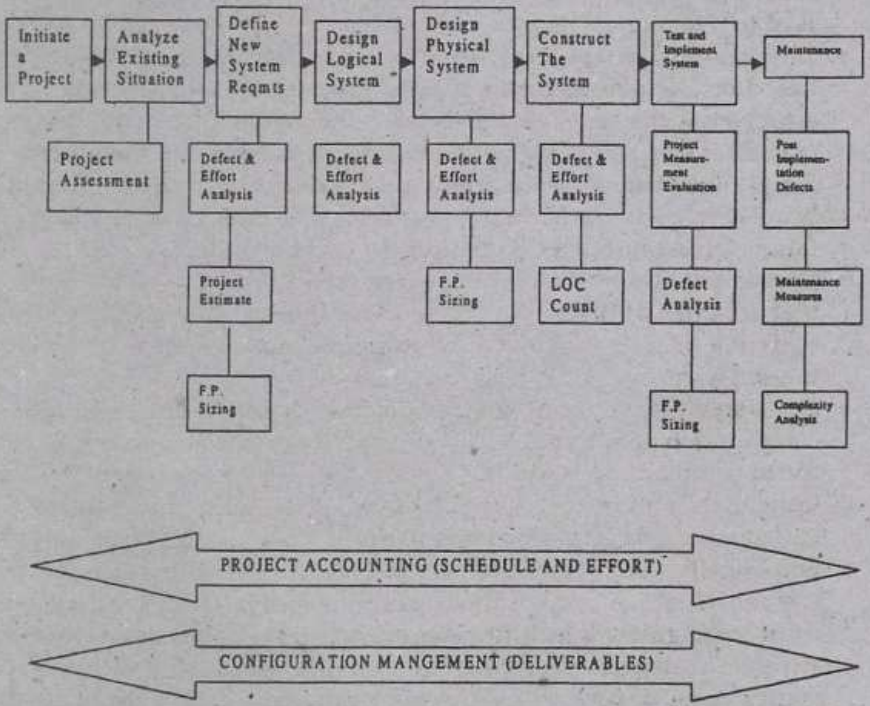

Figure 3: Measurement Activities and the Software Life Cycle

complexity and *defect injection rates* (Defect injection rate is defined as ratio of in-process and customer reported defects with respect to the Total size of Product in Line of Code or Function Point) of existing software should be analyzed. It is at this point that the normal project cost tracking system should be initialized for enhancements.

5.3.1.1 Measure for Product Size

The commonly known units for measuring the size of the software work product are either the lines of code (LOC) or the function points or feature points (FP). For the concept of function point and feature point see the vignette below.

Function Oriented Metrics—Function Points and Feature Points

For measuring the functionality delivered by the application, it makes sense to use function oriented software metrics. This is because "functionality" cannot be measured directly. Based on this concept, function point analysis (FPA) is a one of the popular method for estimating and measuring the size of an application. Function points quantify the functionality provided to the user based on the user's requirements and high-level logical design. Using the FPA method, the size of an application system's functionality is calculated in terms of Function Point count.

Function oriented metrics were first proposed by Allan Albrecht of IBM. He suggested a measure called the *function point*. Function points are derived using an empirical relationship based on countable (direct) measures of software's information domain and assessment of software complexity.

A slight variant over function point is "feature point". According to Capers Jones [1], for applications in which the number of algorithms is uncertain or in which algorithmic factors are not significant, *function points* would be the appropriate choice for a metric. Many business applications fall within this category—for example, accounting software, Sales Information Systems, Marketing Support Systems. For applications in which the number or algorithms is countable, and in which algorithmic factors are significant i.e. high algorithmic complexity, *feature points* would be appropriate choice for a metric. Many scientific, engineering, and real time systems application, process control systems and embedded software fall in this category.

For an easy to understand examples of how the function points are calculated, readers may like to see [14] in the list of references given at the end of this chapter.

For new projects, the requirements phase is the normal point at which the project cost tracking system should be initialized. It is also appropriate to initialize the project defect and quality tracking system, since requirements problems are often a major source of both expense and later troubles.

As the requirements are refined, the next aspect of measurement deals with whether the project should be in the form of a customer application or in the form of a package that is acquired and modified (make v/s buy decision). The risks and value of both approaches are considered.

5.3.2 Measurement during Construction Phase

If the decision is to construct the project, then, a second estimate should be prepared in conjunction with the logical design of the application. Since from this point on defect removal can be the most expensive element, it is imperative to utilize reviews and inspection (this is discussed in Chapter 6) and record defect data. A second and more rigorous cost estimate should be prepared at this time. During physical design, review and inspections are also valuable and defect data will continue to accumulate.

The coding or construction phase of a project can be either troublesome, or almost effortless depending upon the rigour of the preceding phases. A third formal cost estimate should be prepared; it will be very rigorous in accumulating costs to date and very accurate in estimating costs to the completion of the project. Defect and quality data recording should also be kept during code reviews or inspections. Complexity measures of the code itself can now be performed well.

5.3.3 Measurement during Testing Phase

The *testing* phase of a software development project can range from a simple unit test by an individual programmer to a full multi-state formal test suite that includes function test, integration test, stress test, regression test, independent test, field test, system test, and final acceptance test (also the alpha and beta tests—various types tests and test related measures are discussed in Chapter 10). Both the defect data and the cost data from the testing phase should be measured in detail and then analyzed for use in subsequent *defect prevention* activities (see the vignette below on Defect Prevention).

The Principles of Software Defect Prevention

The fundamental objective of software defect prevention is to ensure that defects, once identified and addressed, do not recur. *Everyone* must be involved in defect prevention activities. Tenets of defect prevention are:

- The programmers must evaluate their own errors (common techniques are walkthroughs and inspection discussed in Chapter 6)
- Feedback is an essential part of defect prevention
- There is no panacea (single cure-all that will solve all the problem—an important activity is root-cause-analysis)
- Process improvement must be an integral part of the process
- Process improvement does not happen overnight—there is no silver bullet! It takes all the hard work to improve a process.

Major steps in implementing a defect prevention program are:

- Set up a defect reporting system
- Perform root-cause analysis
- Develop a focused action plan
- Implement the action plan
- Track performance improvement
- Take the lessons learnt and start the cycle all over again!

During the maintenance and enhancement phase, both *user satisfaction measures* and *defect measures* should be carried out. User satisfaction is a basic metric for organizations that market software. It is an important metric for internal information systems too. There are various methods available for measuring user satisfaction; however a discussion on that is not within the scope of this book. It is interesting to note that industry data shows a strong observed co-relationship between defect levels and user satisfaction. From this point of view, measuring software defect rates becomes important. However, before we undertake this discussion, we wish to mention here that often, the terms "bug", "error", "defect" get used interchangeably. Also, sometimes, these terms get used synonymously with other terms like "failures" and "problems". Let us understand these terms before going ahead with the discussion on defect metrics. Watts Humphrey, in his famous book "Managing the Software Process" provides a good discussion on this. The vignette below explains these terms. This discussion is only for the conceptual clarity. In this chapter we use the term defect as the most general one.

Errors, Defects, Bugs, Fault, Failure, Problems

Errors – These are human mistakes. Their quantification would depend on an understanding of the programmer's intentions. (Examples: Typographical and syntactical errors)

Defects—These are improper program conditions that are generally the result of one or multiple errors. All errors *may not* produce software defects (Examples: incorrect comments or minor documentation errors). Conversely, a defect could result from such non-programmer causes like improper packaging or handling of the software product. A typical example would be error in the logo of the splash screen coming up on product start up!

Bugs—A bug or fault is program defect that in encountered while operating the product either under test or while in use. Point to note is that the operational condition or operating environment is often responsible for manifesting a bug which could otherwise have remained un-exposed! Thus, bugs result from defects but all defects do not cause bugs (some are latent and may never be found!) With widely used software (such as commercial off the shelf products) the defects may be found many times, resulting in duplicates or multiple bugs per defect.

Failures—A failure is a malfunction of a user's installation. It may result from a bug, incorrect installation, a communication line fault, a hardware failure, incorrect setting of environmental parameters for the software product etc.

Problems—Problems are user-encountered difficulties. They may result from failures, misuse, or at times, even from misunderstanding!

5.4 Defect Metrics

With this background as discussed above, various *defects measures* and *metrics* are indicated below:

5.4.1 Defect Density

Intrinsic product quality (Chapter 4 has a discussion on *product quality*) is usually measured by the number of "bugs" (functional defects) that a software product may have. Two metrics typically used in this context are *"mean time to failure"* (MTTF) and *defect density*. The MTTF metric is most commonly used with safety critical systems such as airline traffic control, avionics and weaponry applications in military. For example a specification such that air traffic control system *cannot be unavailable* for more than five seconds in a year (i.e. almost nil downtime required). In contrast, the defect density metric is typically used

in commercial software system. Defect density is monitored at different stages as follows: (the terms system testing, acceptance test are explained in Chapter 10).

5.4.1.1 Defect Density (at System Testing Stage)

This is calculated as:

$$\frac{\text{[Total number of Defects identified during System Testing]}}{\text{Actual Size of the product}}$$

where size may be indicated by a suitable measure—most typically either as lines of code (LOC) or Function Points/Features Points (FP)

5.4.1.2 Defect Density (at Acceptances Testing Stage)

This is calculated as:

$$\frac{\text{[Total number of Defects reported at Acceptance Testing]}}{\text{Actual Size of the product}}$$

Though the two metrics (MTTF and Defect Density) are correlated, they are different enough to merit close attention. This is discussed below.

First of all, MTTF show the temporal dimension of the defect—i.e. it measures the *time* between successive failures, whereas, defect density measures the defects relative to the software *size* (lines of code, function points etc.). Secondly, although it is difficult to separate defects and failures in actual measurements and get the data tracked, "failures" and "defects" have different meanings (see the vignette at the end of section 5.3). However, for practical purposes, there is not difference between the two terms. Third, we usually, encounter the defects that cause higher failure rates. The probability of failure associated with a "latent defect" is called its size or "bug size". For missions critical applications (such as air traffic control systems) the operational profile and scenarios are better defined. As such, MTTF metrics is more appropriate here. Fourth, from the data gathering perspective, time between failure data is expensive. . To be useful, time between failure data requires a high degree of accuracy. Gathering such data requires special arrangements and that is not always easy. Finally, the *defect density metric* has another appeal for the development organization of commercial products. Study of such data gives them an opportunity to strengthen/improve their testing processes.

To sum up, regardless of their differences and similarities, MTTF and defect density are two key metrics for intrinsic product quality and as such they are among the most important defect metrics.

5.4.2 Defect Rate

The general, the concept of "defect rate" is the expected number of defects over a certain time period specified is important for cost and resource estimates of the maintenance phase of the software life cycle. It should be noted that "defect rate" and "defect injection rate" (discussed below) are not the same.

5.4.2.1 Defect Injection Rate

Defect injection rate (DIR) is defined as the ratio of in process and customer reported defects with respect to the total size of the product. Thus, DIR is:

$$\frac{[(\text{Number of In-process-defects}) + (\text{Number of Customer-reported Defects})]}{\text{Actual size of the product}}$$

There is an important point to note: *Unit Testing, Integration Testing, System Testing* are the tests carried out by the development organization before the product is shipped. In addition, the work product is reviewed prior to testing. Any defects found in either of these activities i.e. review and/or testing till the product is shipped by the development organization are called the *in process defects*. Once the product is shipped and enters into production stage for actual use, defects reported thereafter are known as the customer-reported defects.

5.4.3 Defect Removal Efficiency

Defect removal efficiency (DRE) is the efficiency of eliminating all defects before delivering the product to the customer. DRE includes all the defects identified and eliminated during review and testing phase (*Review* is discussed in Chapter 6 and *Testing* is discussed in Chapter 10).

DRE is calculated as follows:

$$\frac{\text{Number of defects identified up to System Testing (both during review and testing)}}{[(\text{Number of defects identified up to system testing}) + (\text{number of defects identified at Acceptance testing}) + (\text{number of defects identified during warranty support}) + (\text{Number of defects identified by customer during work product reviews})]}$$

5.4.4 Defect Removal Effectiveness

The discussion here is only for academic interest. In practical scenarios measuring "effectiveness" is almost impossible. However, we discuss

some concepts here only for the sake of completeness of our definitions. The philosophical argument here is that whereas *efficiency* implies the element of time, *effectiveness* is related to the extent of impact and we may feel that the latter is more important.

To define defect removal effectiveness, we must first understand the activities in the development process that are related to *defect injections* (a defect being injected into a product though not intentionally) and *defect removals*. This is illustrated in Figure 4 below

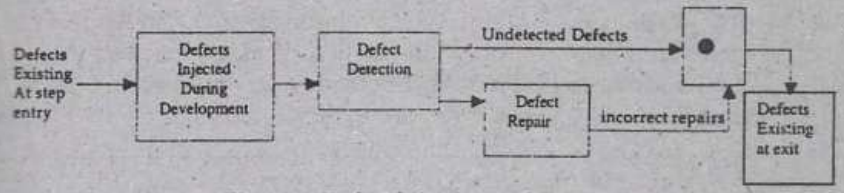

Figure 4: Defect Injection and Removal

Defects are injected (get introduced) into the product or intermediate deliverables of the product (for example, design document) at various phases of software project. It is a wrong assumption to make that all defects are injected at the beginning of the development phase. Before testing, the development activities themselves are subject to defect injection, and the reviews/inspections etc. at the end of the phase activities are the key vehicle for defect removal. For the testing phases, the testing itself is for detecting and reporting defects. When the problems found by testing are fixed incorrectly, this is another chance for error injection. In fact, even for the review/inspection steps, there are the chances for *bad fixes*. Figure 4 depicts the mechanics of defect injection and removal at each step of the development process. From the figure, defect removal effectiveness for each development step can be defined as:

$$\frac{\text{Number of Defects removed (at the step entry)}}{\text{(Number of Defects existing at step entry)} + \text{(Number of Defects injected during development}} \times 100$$

This is the conceptual definition. Note that the number of defects removed is equal to the number of defects detected minus the number of incorrect repairs. If an ideal system existed for defect tracking, all elements in figure 4 could be tracked and analyzed. However, in reality, it is extremely difficult to reliably track incorrect repairs. Assuming that the percentage of incorrect repair or bad fixes is not high, defects removed could be approximated by defects detected. If bad-fix percentage is high (greater than 2% as is the industry practice), one may want to adjust the *defect removal effectiveness* metric accordingly.

5.5 Metrics for Software Maintenance

5.5.1 What is Software Maintenance

Maintenance Projects are important. They are one of the most viable sources of business. It is of enormous importance to the Indian software business scenario where majority of work is in the maintenance and support area. Given that this is such an important area, it would be worthwhile to have a brief discussion on the topic of software maintenance before we discuss the maintenance related metrics. (In Appendix C we have provided information on the history of maintenance process and various maintenance models and maintenance categories.) So let us first understand what software maintenance is.

What is Software Maintenance?

Part of the confusion about software maintenance relates to its very definition. It is difficult to get people agree on a standard definition for "software maintenance". People often wonder what *really* happens during the maintenance phases of the life cycle. Another thing people wonder about is where does it fit into the system/software development life cycle? So let us look at some generally accepted traditional definitions of *software maintenance*:

1. *".....changes that have to be made to computer programs after they have been delivered to the customer or user."*
2. *".....the performance of those activities required to keep a software system operational and responsive after it is accepted and placed into production."*
3. *"Maintenance covers the life of a software system from the time it is installed until it is phased out."*
4. *"Modification of a software product after delivery to correct faults, to improve performance or other attributes, or to adapt the product to a modified operating environment."*
5. *".....software product undergoes modification to code and associated documentation due to a problem or the need for improvement. The objective is to modify existing software product while preserving its integrity."*

From this, one thing is clear: the common theme of above definitions is that *maintenance is an "after-the-fact" activity*. Based on these definitions, no maintenance activities occur during the software/system development effort. *Maintenance occurs after the product is in operation* (during the post delivery stage).

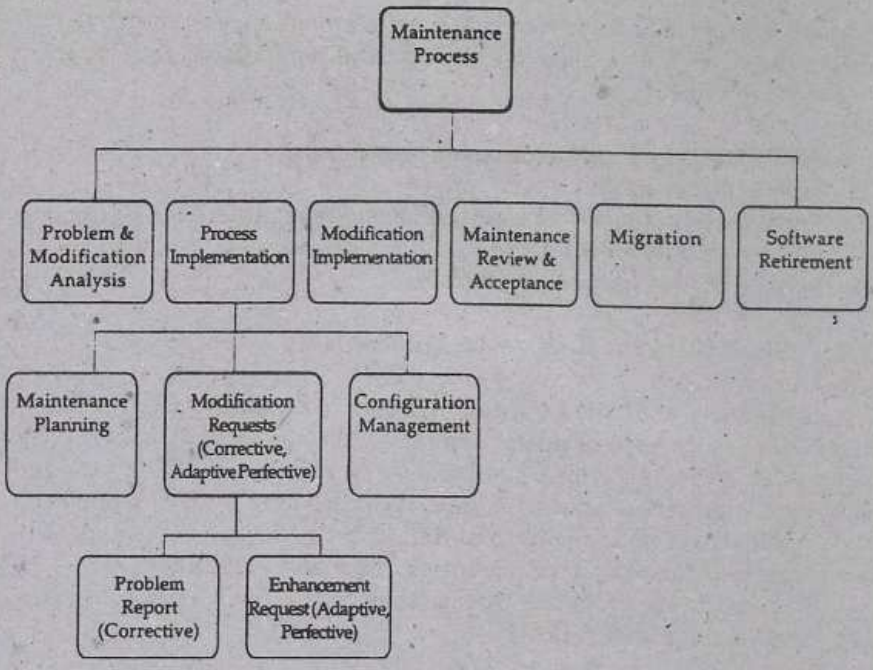

Figure 5: Maintenance Process Activities

5.5.2 Activities Involved in Software Maintenance

Having seen what software maintenance is, let us understand the activities involved in software maintenance. This will form a good background for the discussion on maintenance metrics. Figure 5 above depicts the activities involved in a software maintenance process.

5.5.3 Metrics for Software Maintenance

As seen in the light of discussion above, when a software product has completed its development and is released to the market, it enters into the maintenance phase of its life cycle. During this phase, the defect arrivals by time interval and customer problem calls (which may or may not be defects) are the de facto metrics. However, the roots of these defects or problems reported are in the development process prior to the maintenance phase. Not much can done to alter the quality of the product during the maintenance phase. Therefore, these two de facto metrics, while important, do not truly reflect the quality of software maintenance. All that can be done during the maintenance phase is to fix the defects at the earliest possible and with excellent fix quality (i.e. minimum Bad fix—this term will be explained later in this section). Such actions, although still not able to improve the defect rate (the term